

tourmaline
core

TDD на бекенде

Александр Шинкарев

Test-Driven Development (TDD) разработка через тестирование — это когда сначала тесты потом код. Всё так просто.....

Назад в 90-е



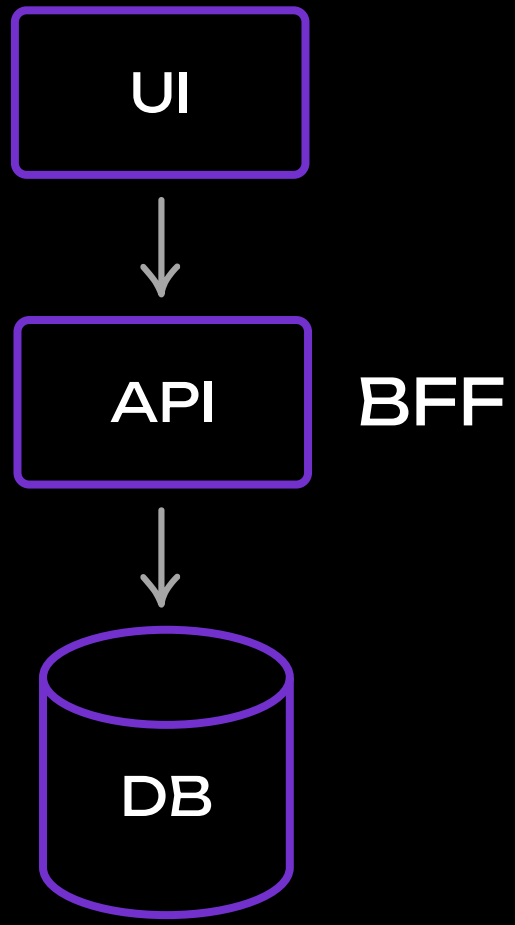
Кадр из к/ф «Идиократия» (Idiocracy), реж. Майк Джадж, 2006 г. Правообладатель: 20th Century Studios.

Любимое To Do приложение

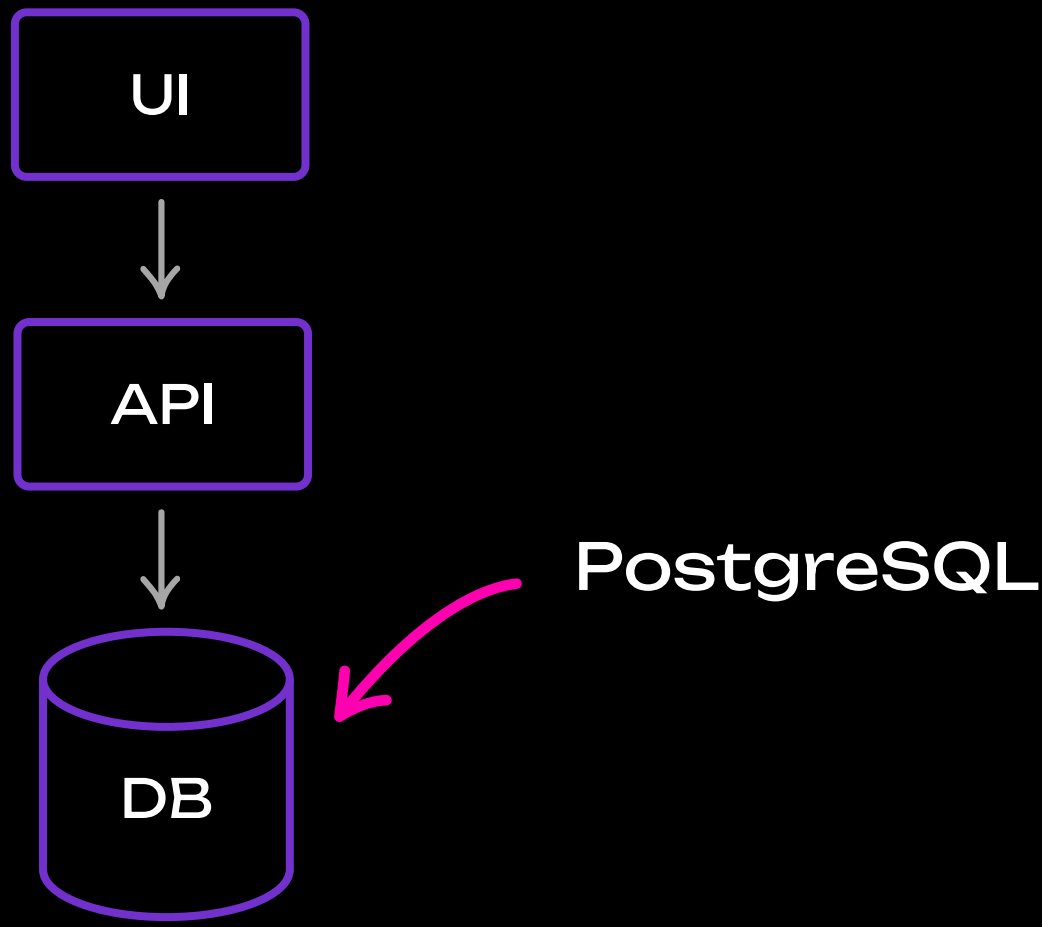
- ✓ Много простого кода на C#
- ✓ Большой контекст на удержание
- ✓ Авторский (спорный) подход
- ✓ Всё будет доступно в 2 открытых репозиториях



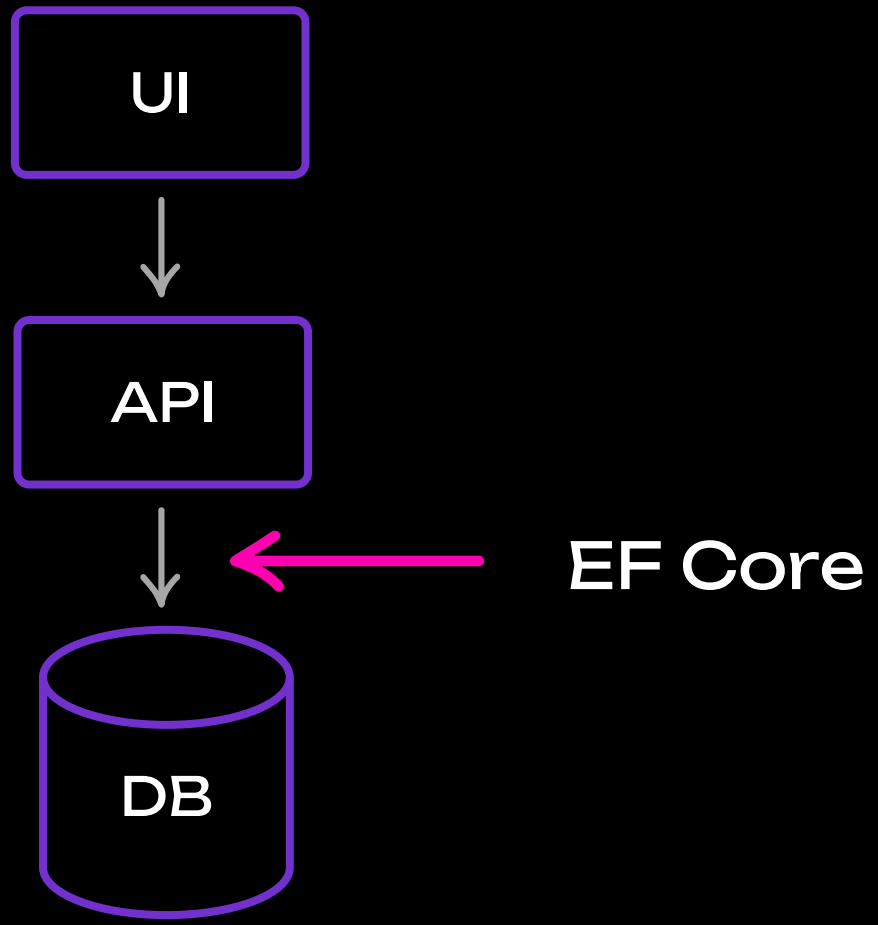
Архитектура и из чего сделаны наши бекенды



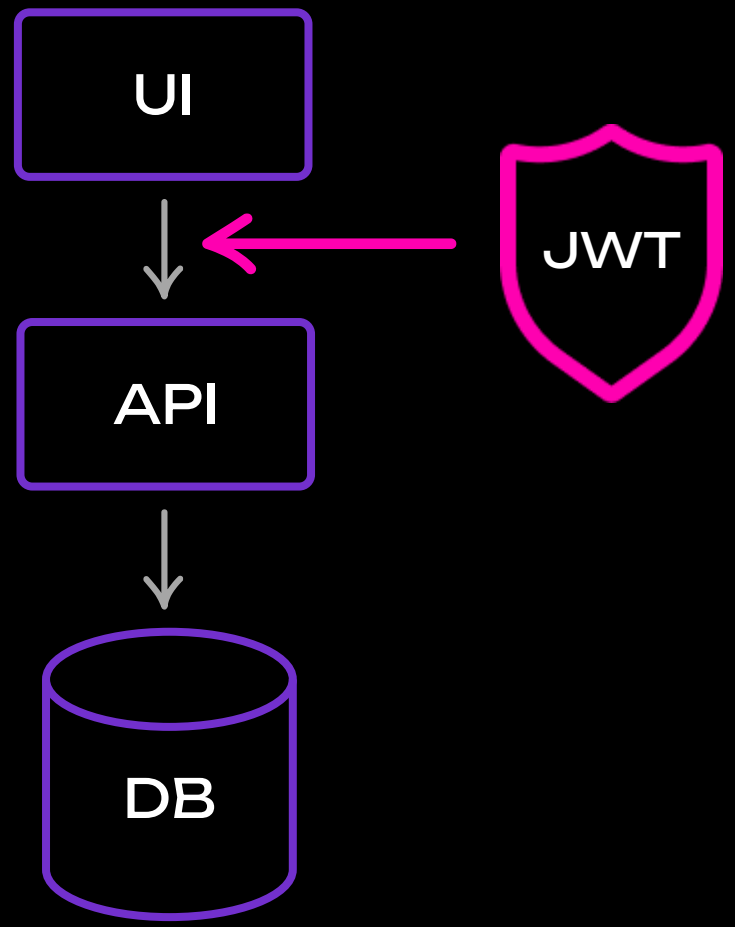
Архитектура и из чего сделаны наши бекенды



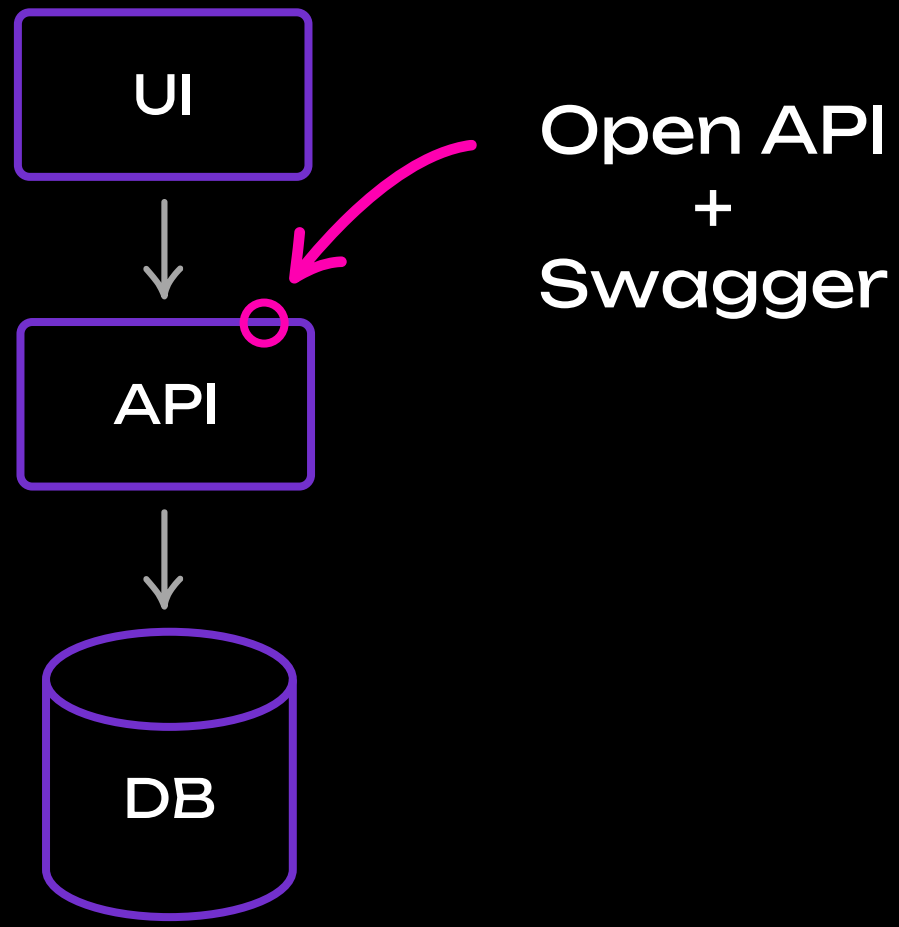
Архитектура и из чего сделаны наши бекенды



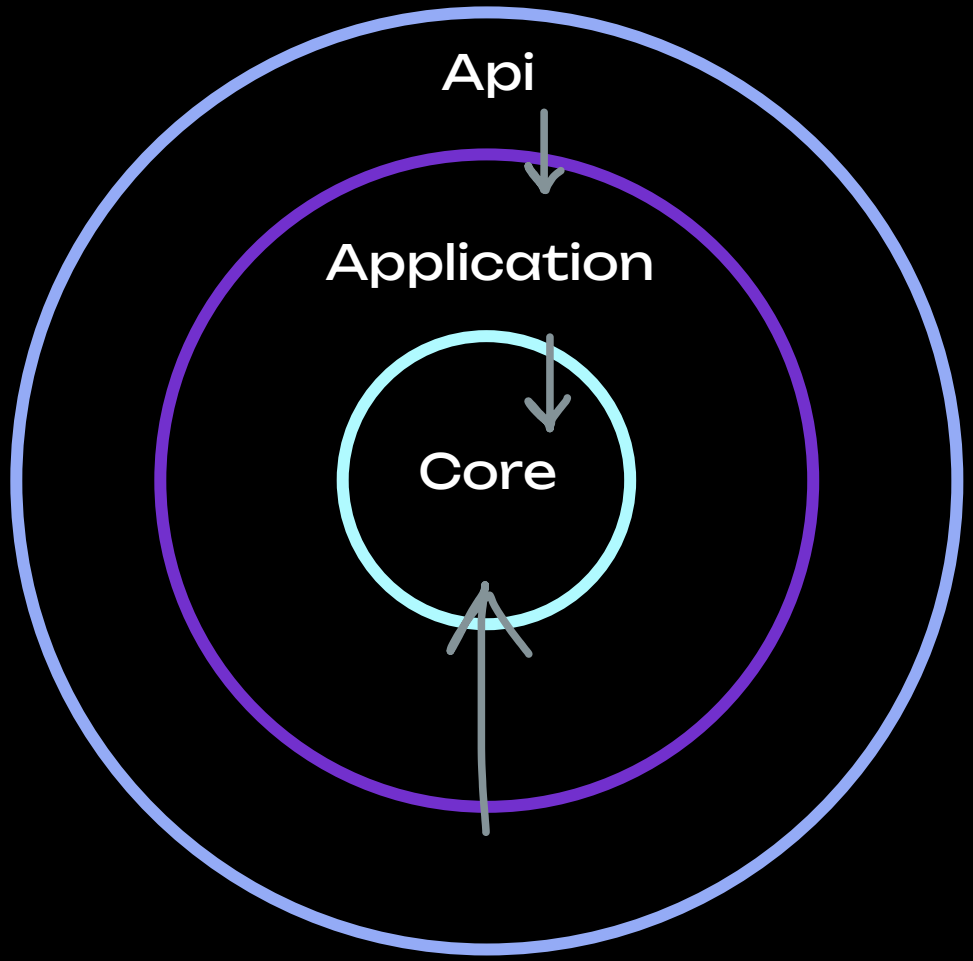
Архитектура и из чего сделаны наши бекенды



Архитектура и из чего сделаны наши бекенды



Слои



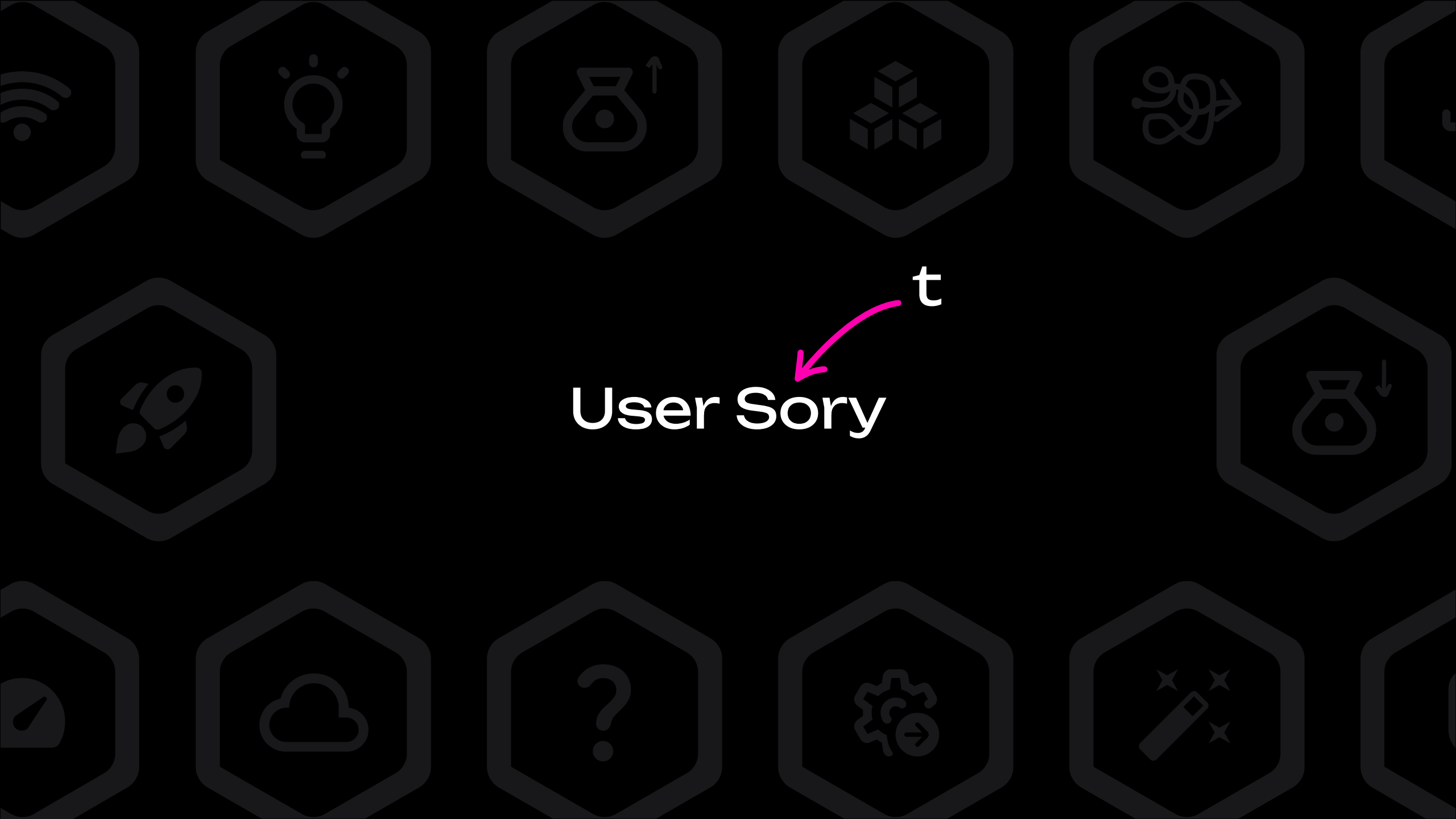
Процесс

User Story

Контракт

Тестовая стратегия

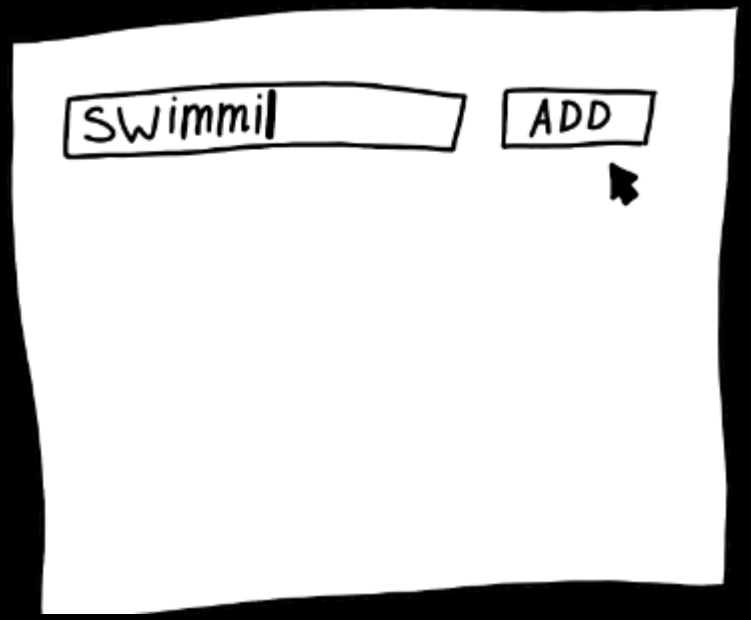
TDD


t
User Sory

Новая To Do

Хочу создавать новые To Do с именем

Чтобы я мог отслеживать свои дела



Список To Do

Хочу видеть мои To Do от самых новых до самых старых

Чтобы знать что ещё осталось сделать



Контракт на пальцах

Эндпоинт создания

POST /to-dos

Request Body

```
{
  "name": "Dishes"
}
```

Response

```
{
  "newToDoId": 2
}
```

Эндпоинт на чтение

GET /to-dos

Response

```
{
  "toDos": [
    {
      "id": 1,
      "name": "Test"
    }
  ]
}
```

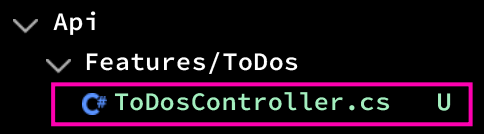
Эндпоинт удаления

DELETE /to-dos?toDoId=2

Response

```
{
  "isDeleted": true
}
```

Контракт на C#




```
[ApiController]
[Route("to-dos")]
public class ToDosController : ControllerBase
```

Дихотомия добра и зла DTO

Request

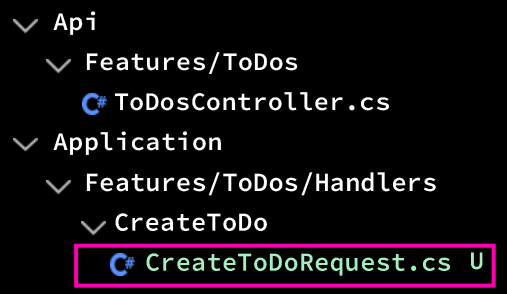
- Тело запроса

Response

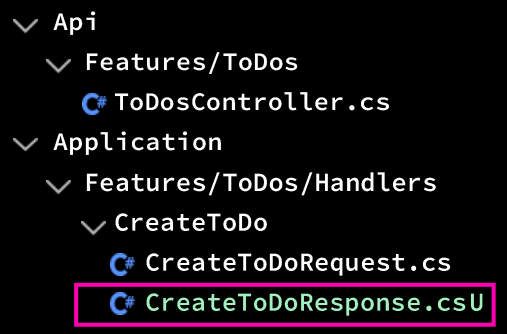
- Ответ сервера



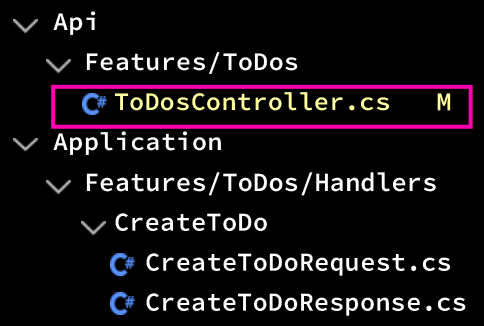
Кадры из к/ф «Пираты Карибского моря: Сундук мертвеца» (Pirates of the Caribbean: Dead Man's Chest), реж. Гор Вербински, 2006 г. Правообладатель: Walt Disney Pictures.



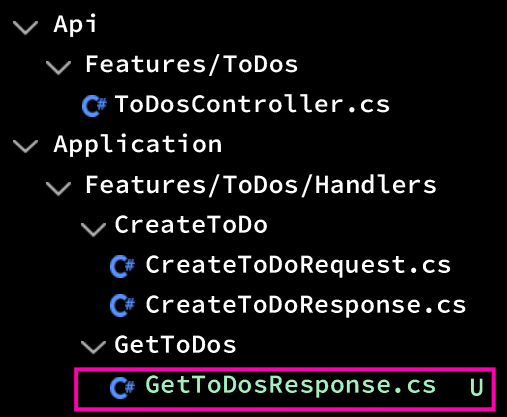

```
public class CreateToDoRequest
{
    public required string Name { get; set; }
}
```



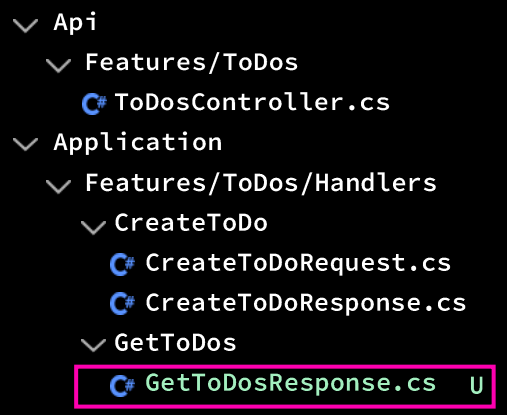

```
public class CreateToDoResponse
{
    public required long NewToDoId { get; set; }
}
```




```
[HttpPost]
public Task<CreateToDoResponse> CreateToDoAsync(
  [Required] [FromBody] CreateToDoRequest createToDoRequest
)
{
  throw new NotImplementedException();
}
```

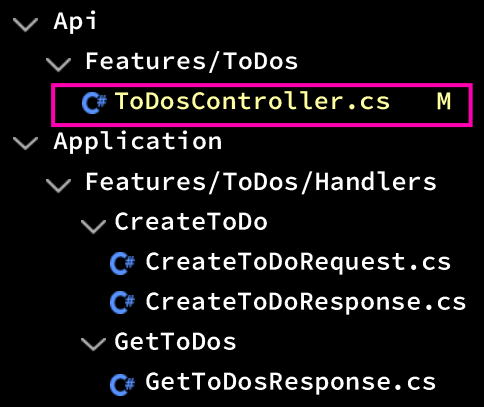



```
public class GetToDosResponse
{
    public required List<ToDoDto> ToDos { get; set; }
}
```

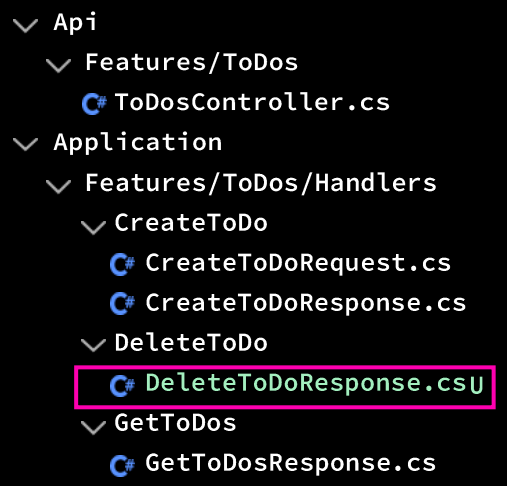



```
public class ToDoDto
{
    public required long Id { get; set; }

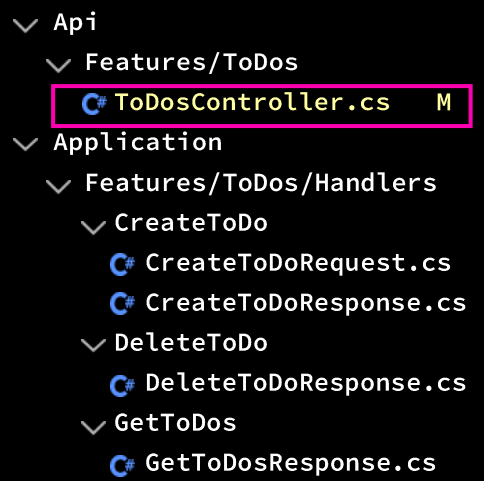
    public required string Name { get; set; }
}
```




```
[HttpGet]
public Task<GetToDosResponse> GetToDosAsync()
{
    throw new NotImplementedException();
}
```




```
public class DeleteToDoResponse
{
    public required bool IsDeleted { get; set; }
}
```

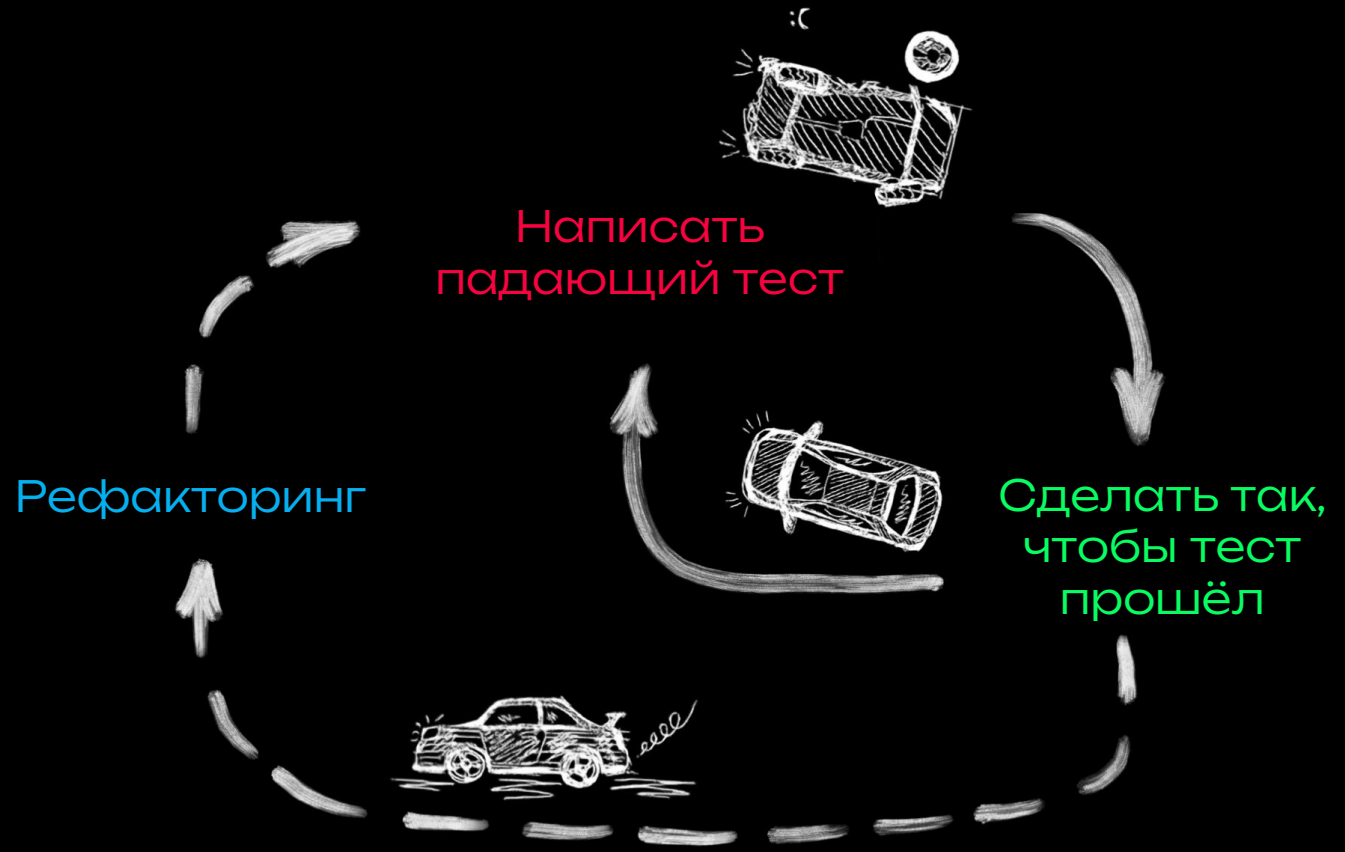



```
[HttpDelete]
public Task<DeleteToDoResponse> DeleteToDoAsync(
  [Required] [FromQuery] long toDoId
)
{
  throw new NotImplementedException();
}
```

Тестовая стратегия

Памятка по TDD

Красный — падает
Зелёный — проходит
Синий — улучшить



E2E-тесты

Принципы наших E2E

- ✓ Покрывают только Happy Path
- ✓ Вызывают каждый эндпоинт хотя бы 1 раз
- ✓ Проверяют, что сохранённое также читается

E2E-тесты на Karate

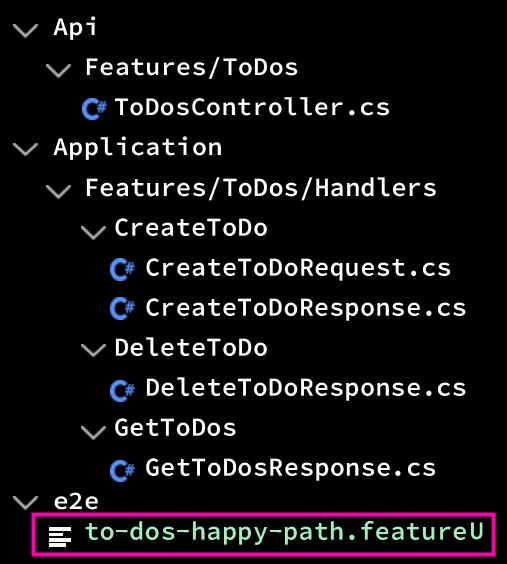
```
Given url apiRootUrl
And path 'swagger/openapi.json’
When method GET
Then status 200
And match response.info ==
"""
{
    "title": "to-dos-api-dotnet"
}
"""
```

1. Декларативные
2. Не зависят от стека бекенда

Наш E2E-сценарий

- ✓ Создадим To Do
- ✓ Прочитаем его
- ✓ Удалим за собой
- ✓ Проверим, что удалилось

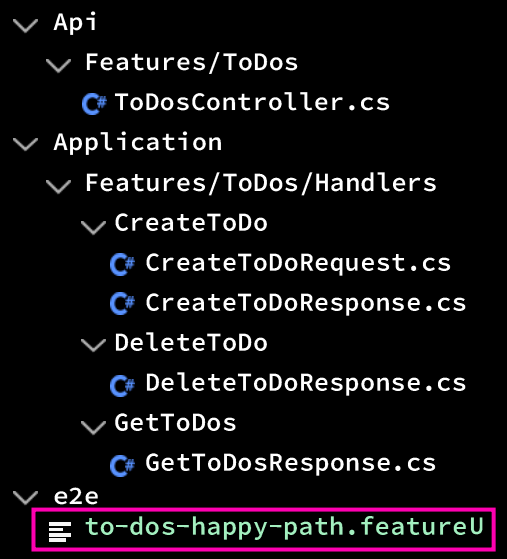




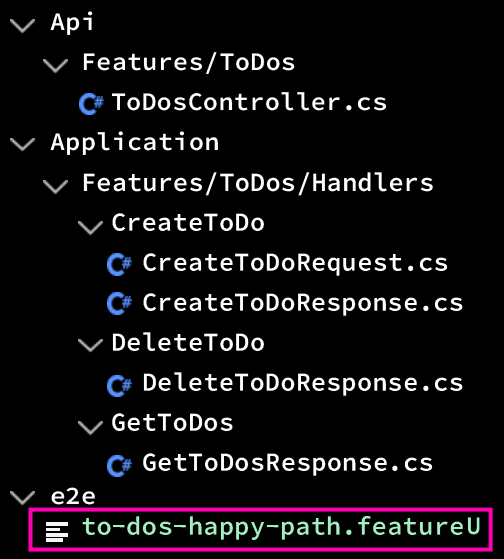

```
* def randomToDoName = '[API-E2E]-to-do-' + Math.random()

Given url apiRootUrl
And path 'to-dos'
And request
"""
{
  "name": "#(randomToDoName)",
}
"""
When method POST
Then status 200

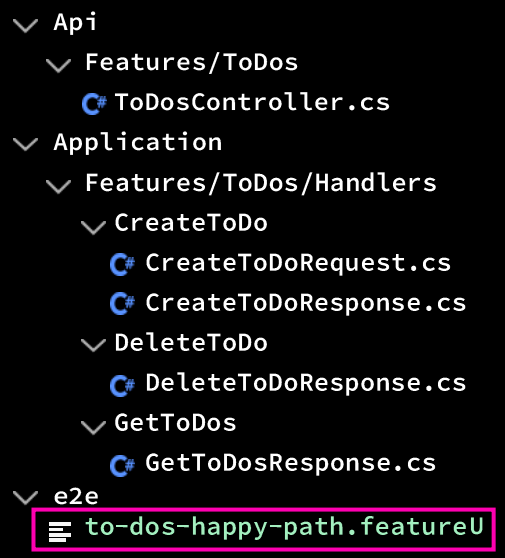

* def newToDoId = response.newToDoId
```



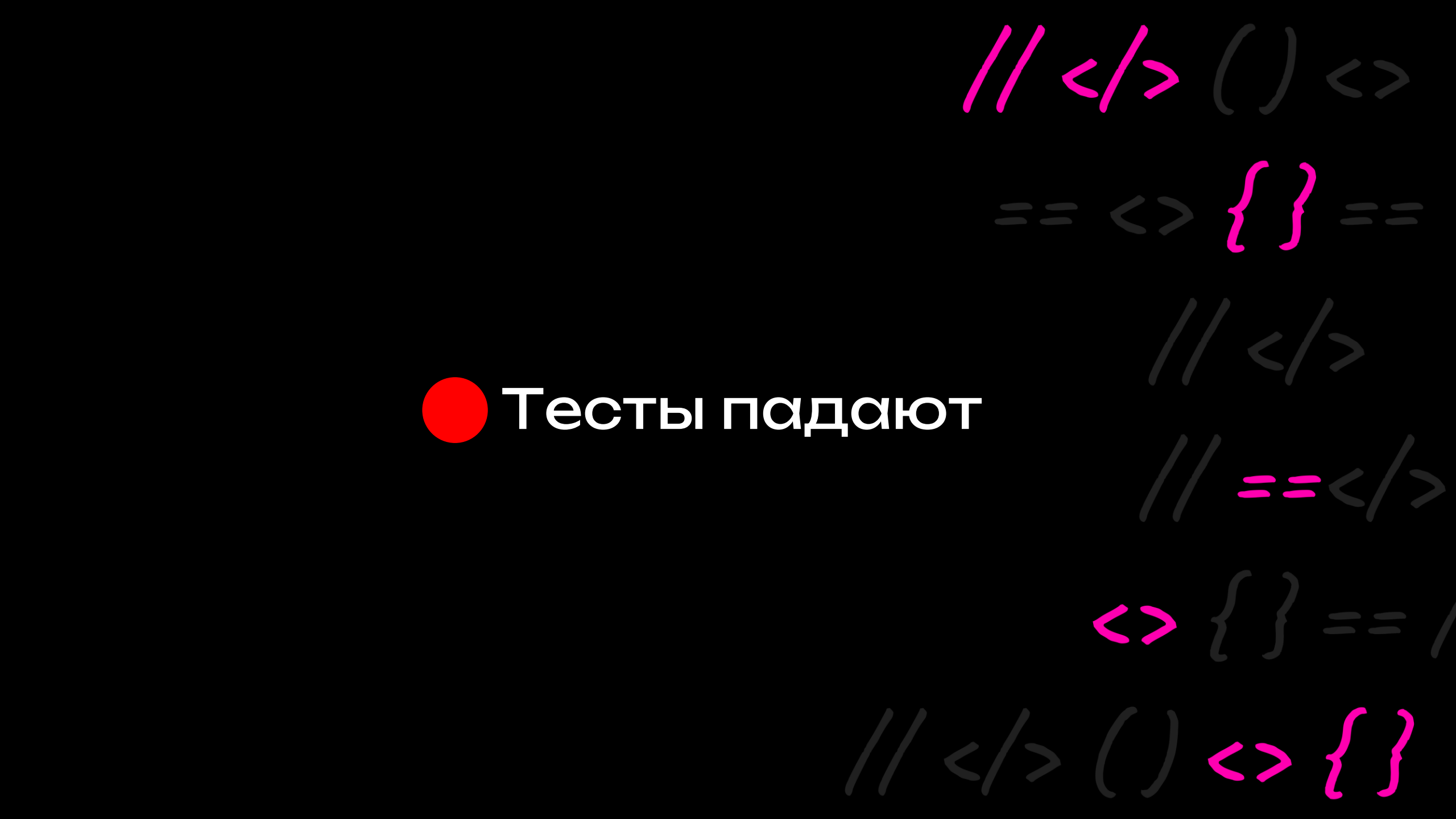

```
Given url apiRootUrl
And path 'to-dos'
When method GET
Then match response.toDos contains
"""
{
  "id": "#(newToDoId)",
  "name": "#(randomToDoName)",
}
"""
```




```
Given url apiRootUrl
Given path 'to-dos'
And params { toDoId: "#(newToDoId)" }
When method DELETE
Then status 200
And match response ==
"""
{
  "isDeleted": true,
}
"""
```




```
Given url apiRootUrl
Given path 'to-dos'
When method GET
Then status 200
And assert response.toDos.every(x => x.id !== newToDoId)
```


Тесты падают

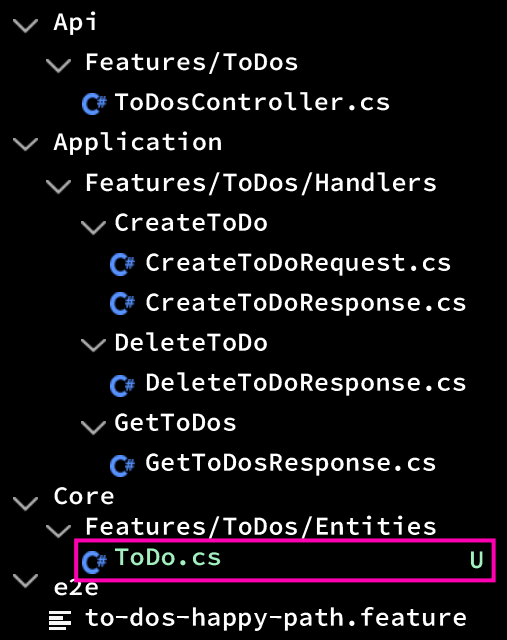
Фейковая реализация (опционально)

Статья М. Фаулера Mocks Aren't Stubs (2007)

- **Fake** objects actually have working implementations, but usually take some shortcut which makes them not suitable for production (an in memory database is a good example).

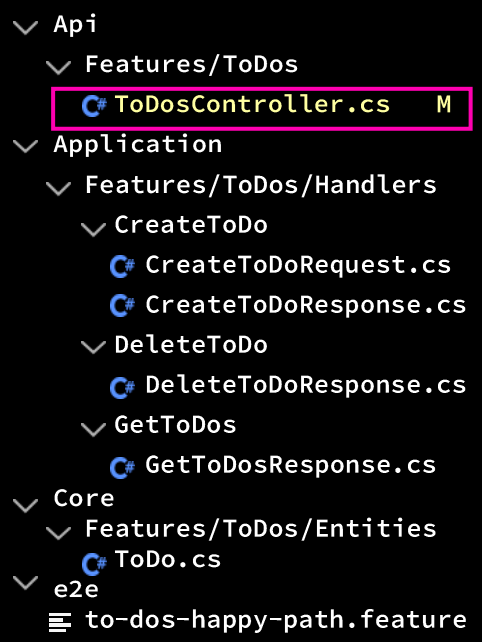


Статья М. Фаулера про моки и заглушки

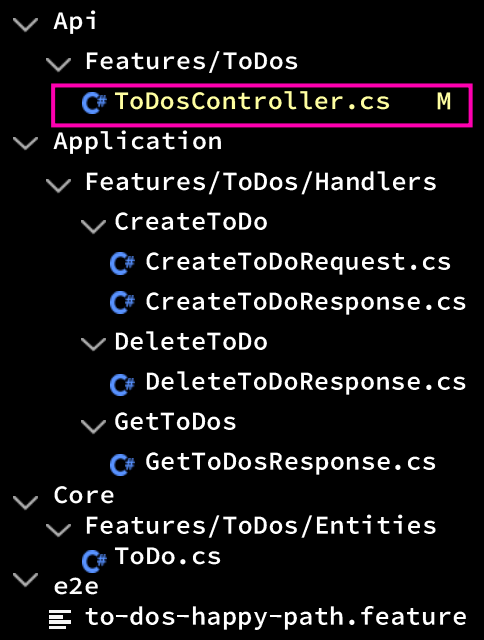



```
public class ToDo
{
  public long Id { get; set; }

  public required string Name { get; set; }
}
```

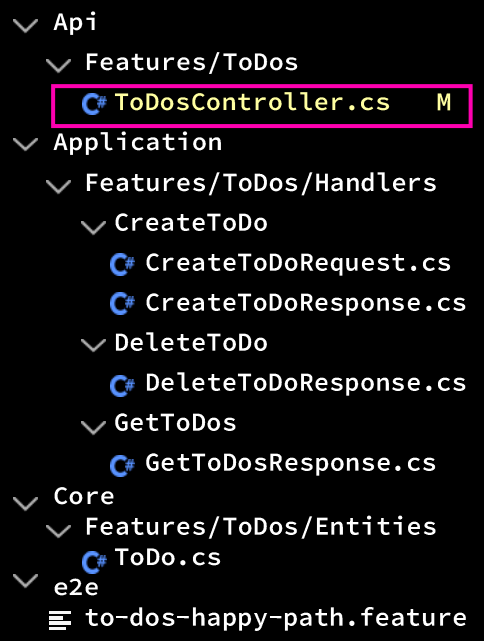



```
public class ToDosController : ControllerBase
{
  private static long _nextToDoId = 0;
  private static readonly
    ConcurrentDictionary<long, ToDo> ToDos = new();
```

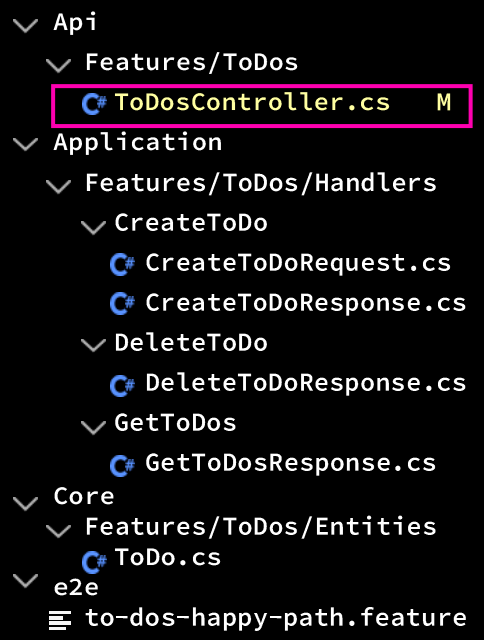



```
[HttpPost]
public Task<CreateToDoResponse> CreateToDoAsync(
  [Required] [FromBody] CreateToDoRequest createToDoRequest
)
{
  var newToDo = new ToDo
  {
    Id = Interlocked.Increment(ref _nextToDoId),
    Name = createToDoRequest.Name,
  };
  ToDos[newToDo.Id] = newToDo;

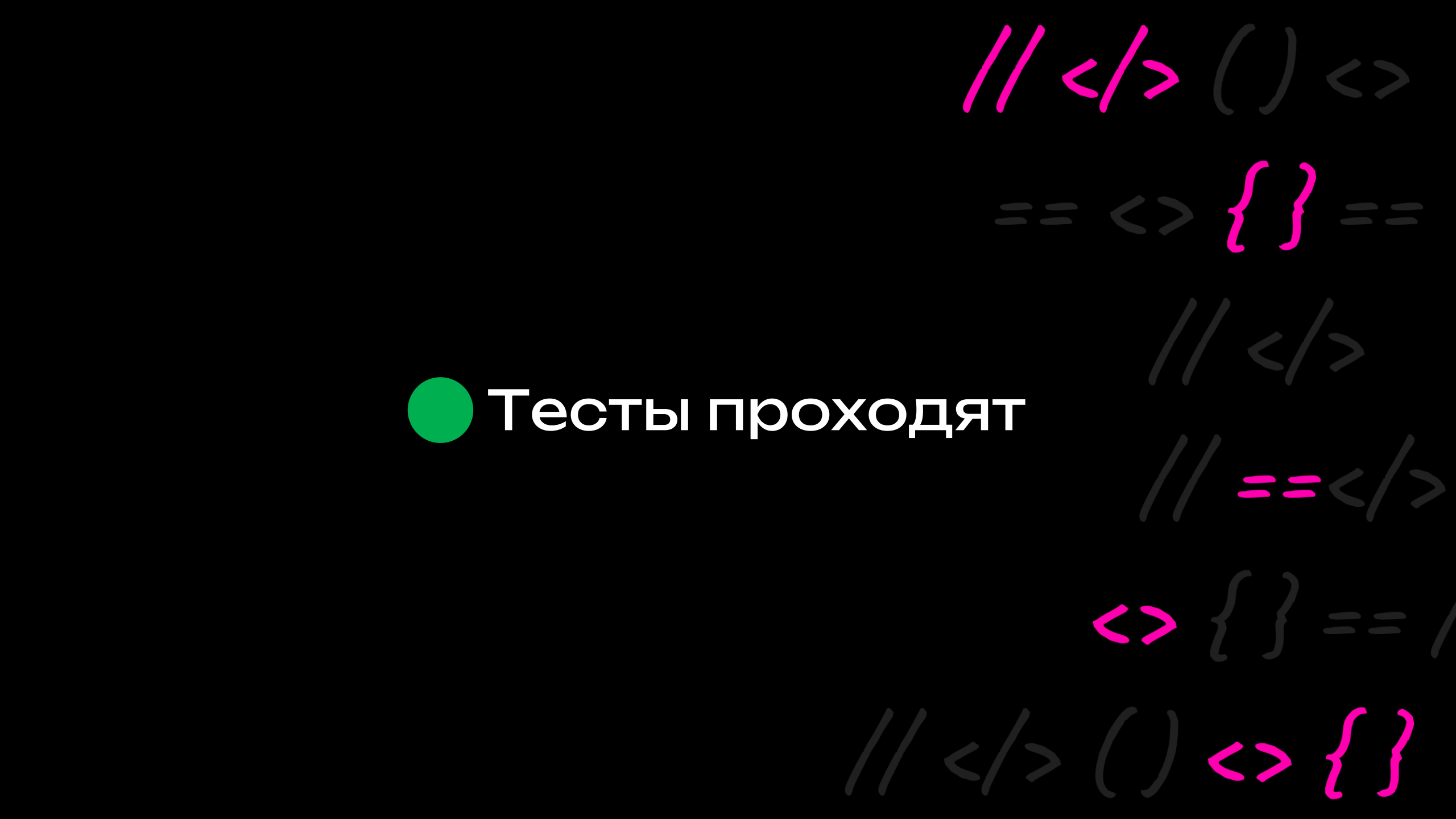
  return Task.FromResult(
    new CreateToDoResponse() { NewToDoId = newToDo.Id }
  );
}
```



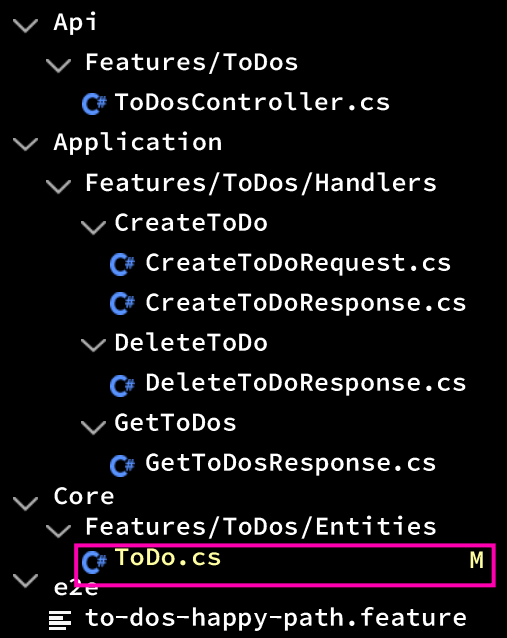

```
[HttpGet]
public Task<GetToDosResponse> GetToDosAsync()
{
  return Task.FromResult(
    new GetToDosResponse
    {
      ToDos = ToDos
        .Values.Select(x => new ToDoDto {
          Id = x.Id,
          Name = x.Name,
        })
        .ToList(),
    }
  );
}
```




```
[HttpDelete]
public Task<DeleteToDoResponse> DeleteToDoAsync(
  [Required] [FromQuery] long toDoId
)
{
  return Task.FromResult(
    new DeleteToDoResponse {
      IsDeleted = ToDos.Remove(toDoId, out _),
    }
  );
}
```


Тесты проходят

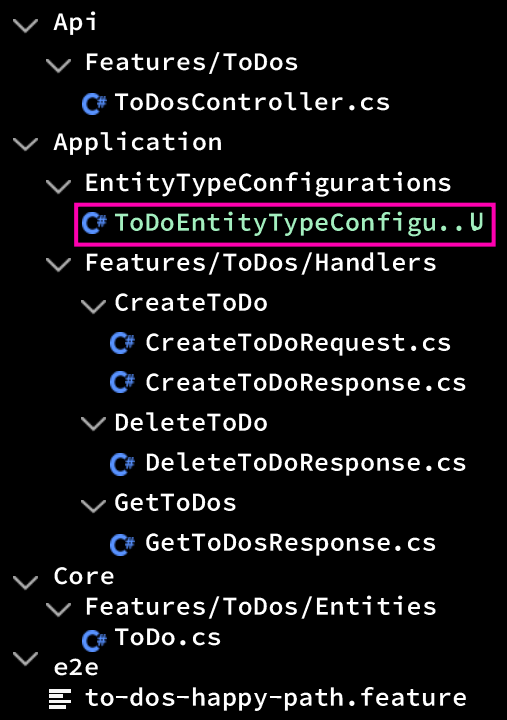
Добавляем базу




```
public class ToDo
{
    public ToDo() { }

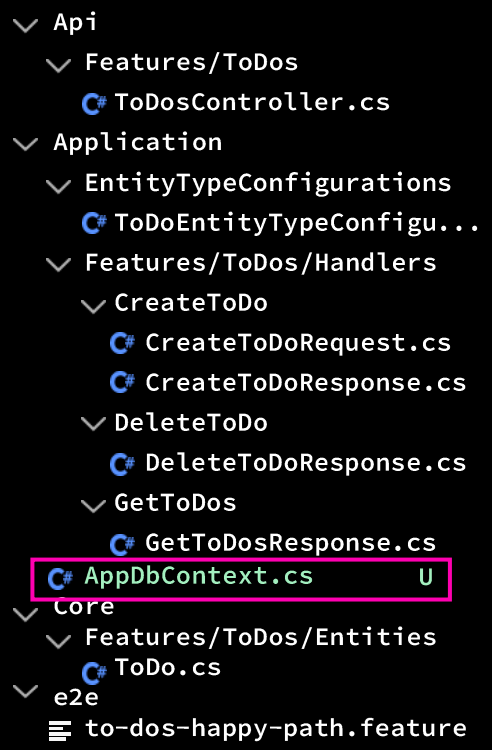
    public long Id { get; set; }

    public required string Name { get; set; }
}
```




```
public class ToDoEntityTypeConfiguration
  : IEntityTypeConfiguration<ToDo>
{
  public void Configure(EntityTypeBuilder<ToDo> builder)
  {
    builder.HasIndex(x => x.Id).IsUnique();

    builder.Property(x => x.Name).IsRequired()
      .HasMaxLength(256);
  }
}
```




```
public class AppDbContext : DbContext
{
  public AppDbContext(
    DbContextOptions<AppDbContext> options
  ) : base(options) { }

  public AppDbContext() { }

  public virtual DbSet<ToDo> ToDos { get; set; }
```

Создаём миграцию

```
dotnet ef migrations add AddToDos
  --startup-project ./Api/Api.csproj
  --project ./Application/Application.csproj
  --context AppDbContext
  --verbose
```

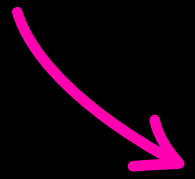
Немного не о CQRS

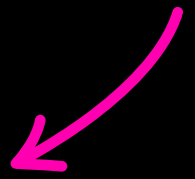
Command

- Меняет состояние в базе (пишет)
- Принимает Request

Query

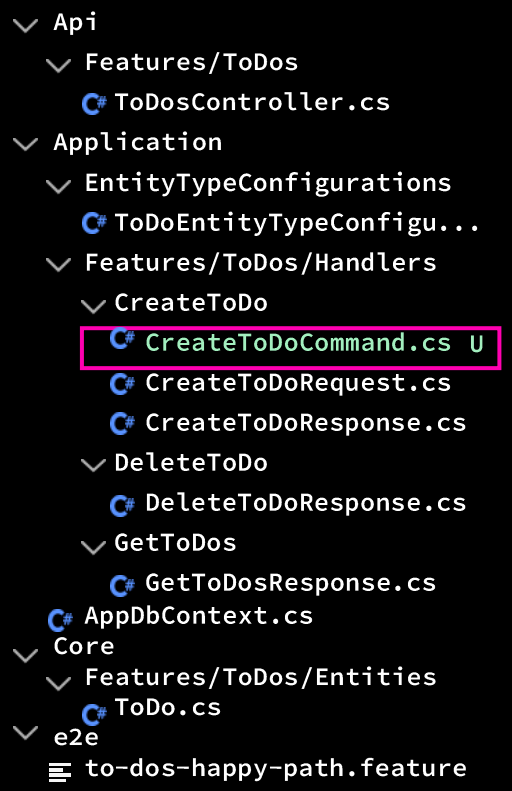
- Читает из базы без модификаций
- Возвращает сущности из Core





Handler

- Оркестрирует команды и квери
- Маппит результат в Response

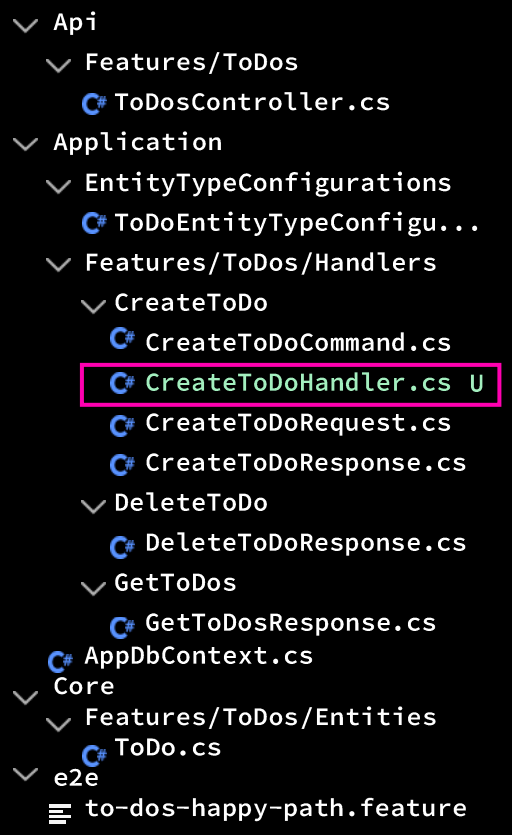



```
public class CreateToDoCommand(AppDbContext context)
{
  public async Task<long> ExecuteAsync(
    CreateToDoRequest createToDoRequest
  )
  {
    var newToDo = new ToDo {
      Name = createToDoRequest.Name,
    };

    await context.ToDos.AddAsync(newToDo);

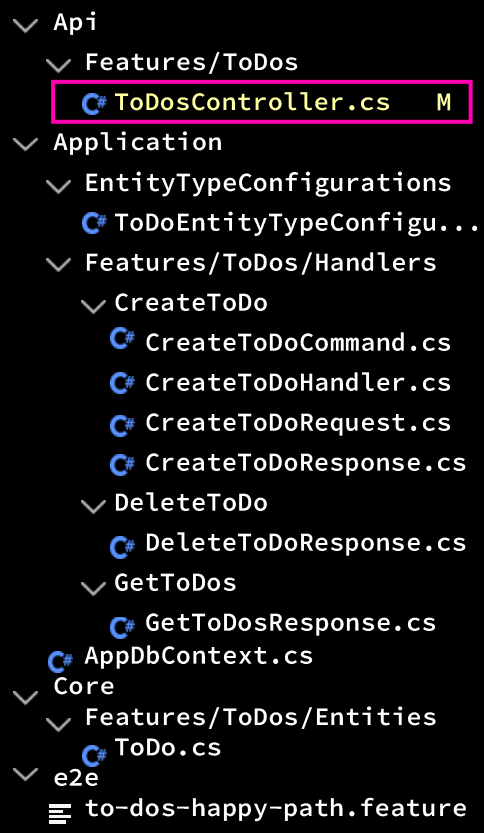
    await context.SaveChangesAsync();

    return newToDo.Id;
  }
}
```

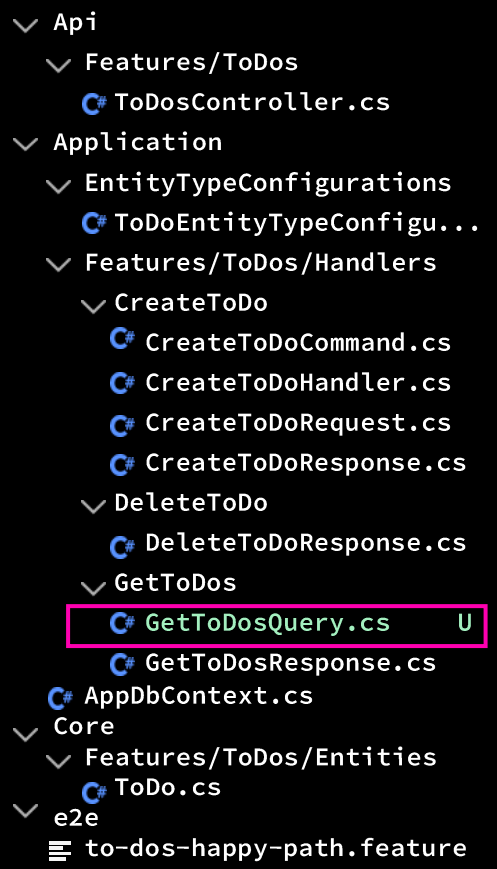



```
public class CreateToDoHandler(
  CreateToDoCommand createToDoCommand
)
{
  public async Task<CreateToDoResponse> HandleAsync(
    CreateToDoRequest createToDoRequest
  )
  {
    var newToDoId = await createToDoCommand.ExecuteAsync(
      createToDoRequest
    );

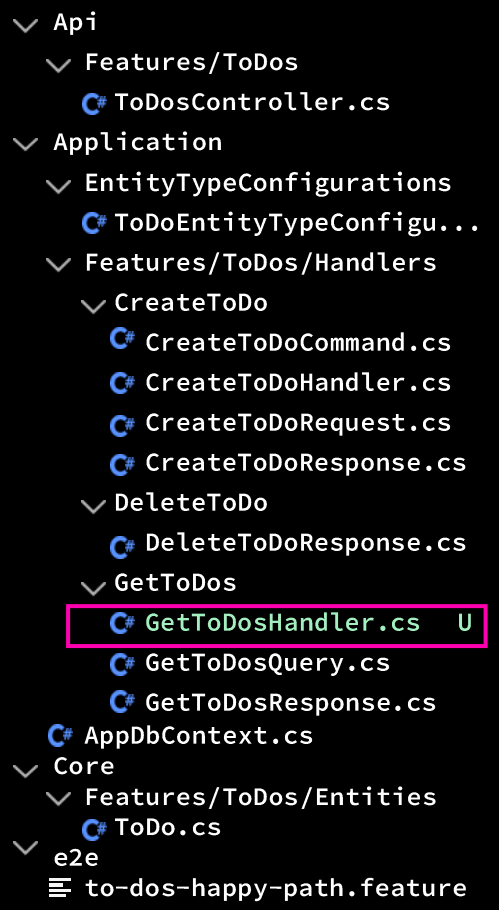
    return new CreateToDoResponse { NewToDoId = newToDoId };
  }
}
```



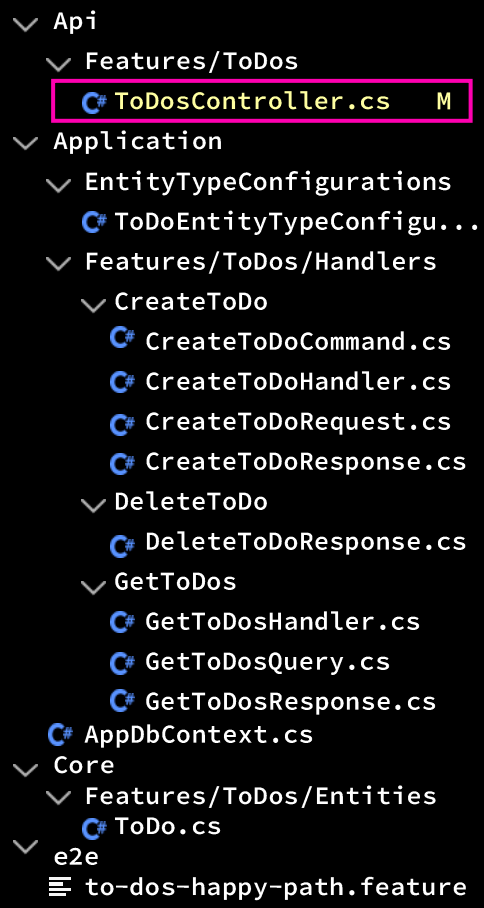

```
[HttpPost]
public Task<CreateToDoResponse> CreateToDoAsync(
    [Required] [FromBody] CreateToDoRequest createToDoRequest,
    [FromServices] CreateToDoHandler createToDoHandler
)
{
    return createToDoHandler.HandleAsync(createToDoRequest);
}
```



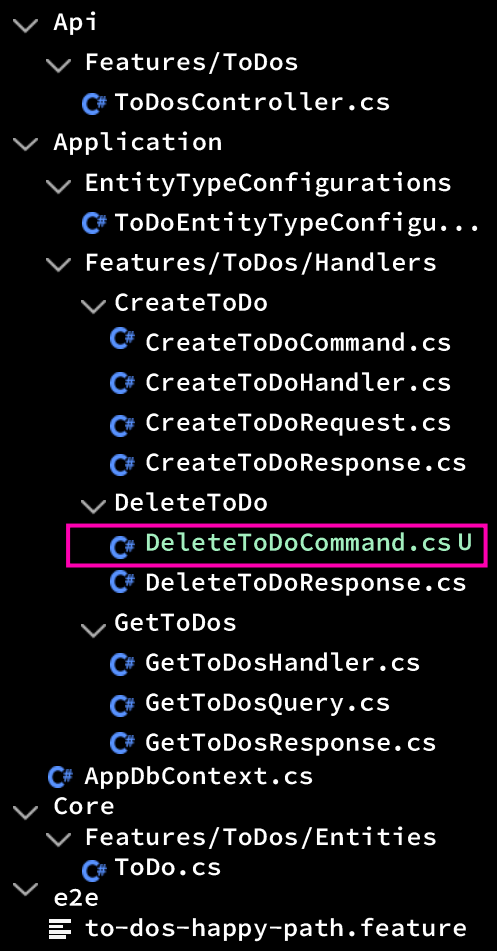

```
public class GetToDosQuery(AppDbContext context)
{
  public Task<List<ToDo>> GetAsync()
  {
    return context
      .QueryableAsNoTracking<ToDo>()
      .ToListAsync();
  }
}
```




```
public class GetToDosHandler(GetToDosQuery getToDosQuery)
{
  public async Task<GetToDosResponse> HandleAsync()
  {
    var toDos = await getToDosQuery.GetAsync();
    return new GetToDosResponse
    {
      ToDos = toDos
        .Select(x => new ToDoDto {
          Id = x.Id,
          Name = x.Name,
        })
        .ToList(),
    };
  }
}
```




```
[HttpGet]
public Task<GetToDosResponse> GetToDosAsync(
  [FromServices] GetToDosHandler getToDosHandler
)
{
  return getToDosHandler.HandleAsync();
}
```

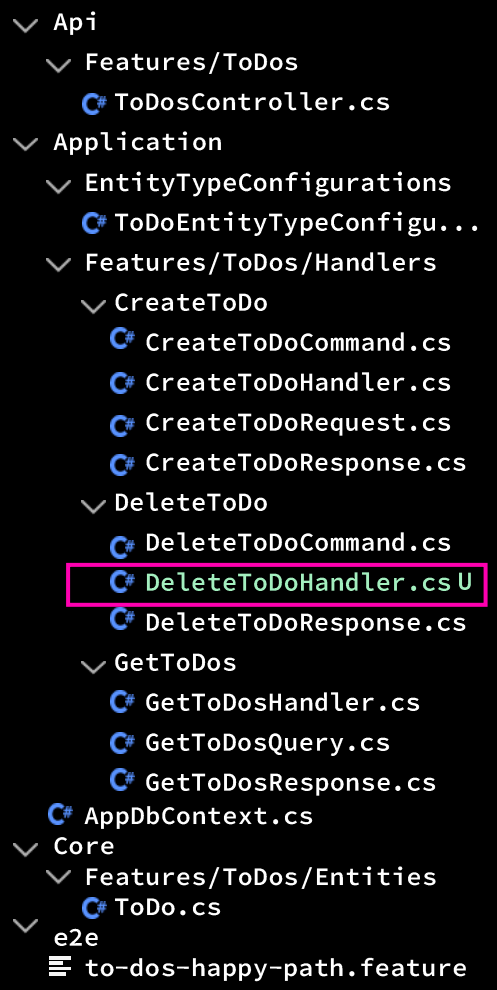



```
public class DeleteToDoCommand(AppDbContext context)
{
  public async Task<bool> ExecuteAsync(long toDoId)
  {
    var toDoToDelete = await context
      .Queryable<ToDo>()
      .SingleAsync(x => x.Id == toDoId);

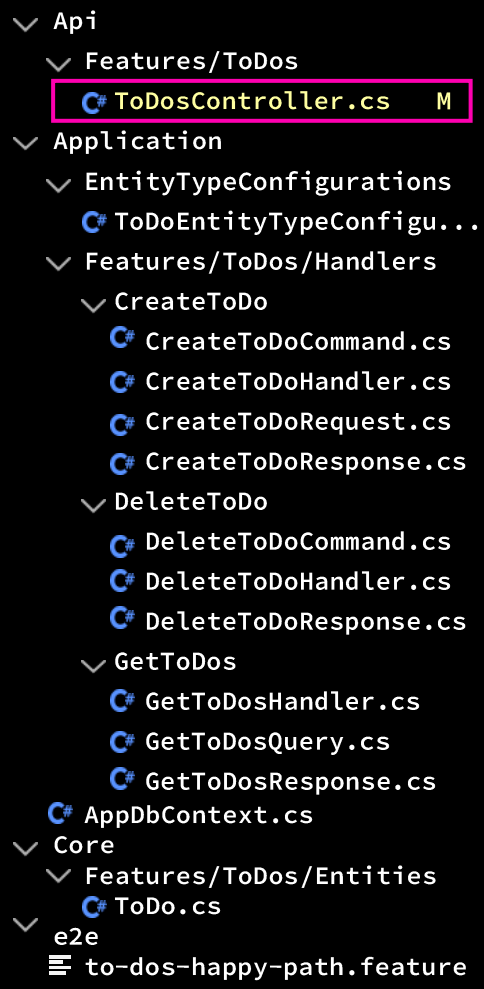
    context.Set<ToDo>().Remove(toDoToDelete);

    await context.SaveChangesAsync();

    return true;
  }
}
```




```
public class DeleteToDoHandler(
  DeleteToDoCommand deleteToDoCommand
)
{
  public async Task<DeleteToDoResponse> HandleAsync(
    long toDoId
  )
  {
    return new DeleteToDoResponse
    {
      IsDeleted = await deleteToDoCommand.ExecuteAsync(
        toDoId
      ),
    };
  }
}
```




```
[HttpDelete]
public Task<DeleteToDoResponse> DeleteToDoAsync(
  [Required] [FromQuery] long toDoId,
  [FromServices] DeleteToDoHandler deleteToDoHandler
)
{
  return deleteToDoHandler.HandleAsync(toDoId);
}
```

- Тесты всё ещё проходят

Список To Do

Хочу видеть мои To Do от самых новых до самых старых

Чтобы знать что ещё осталось сделать



Тестим по настоящей базе

EF Core

EF Core itself contains over 30,000 tests against SQL Server alone

to-dos-api-dotnet

1 тест за 2 секунды
Пайплайн < 2 минут

inner-circle-time-api

106 тестов за 3 секунды
Пайплайн < 2 минут

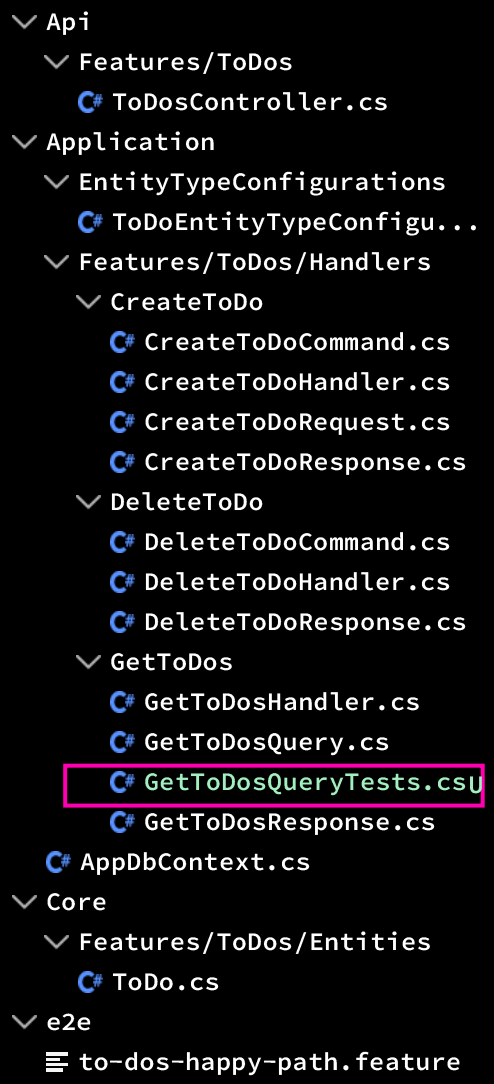
Настоящесть

Параллельность

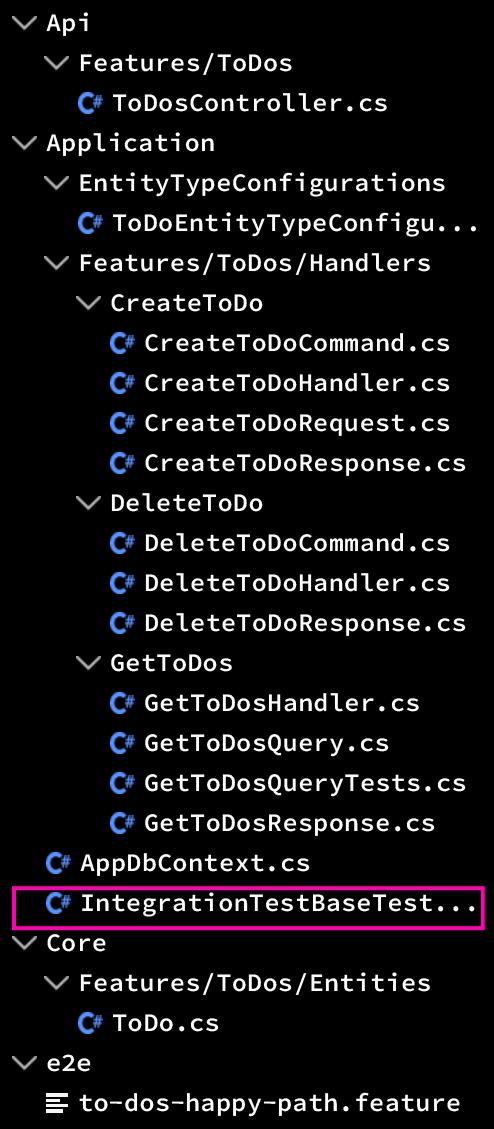
Изолированность



Дока тестировани я от EF Core




```
public class GetToDosQueryTests : IntegrationTestBase
{
  [Fact]
  public async Task GetAsync_ShouldOrderNewerToDosFirst()
  {
    var context = CreateAppDbContext();
```




```
protected AppDbContext CreateAppDbContext()
{
  var context = new AppDbContext(_dbContextOptions);

  context.Database.UseTransaction(_dbTransaction);

  return context;
}
```

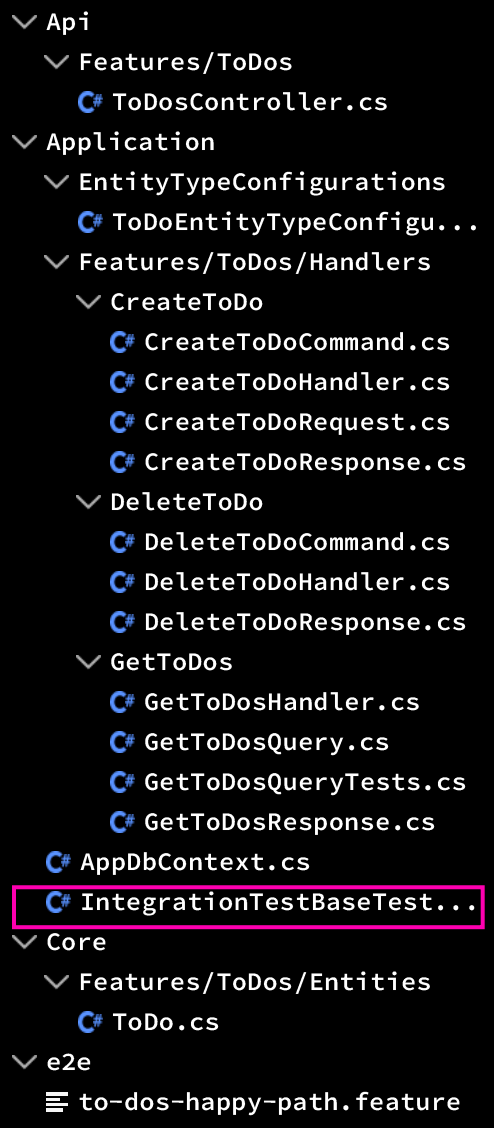



```
_dbConnection = new NpgsqlConnection(connectionString);

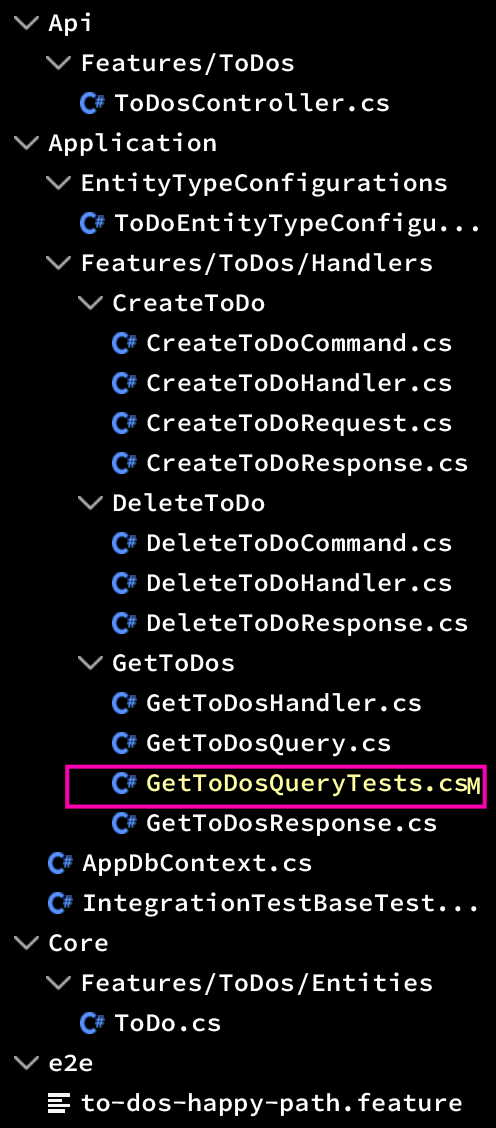
await _dbConnection.OpenAsync();

await ApplyMigrationsAsync();

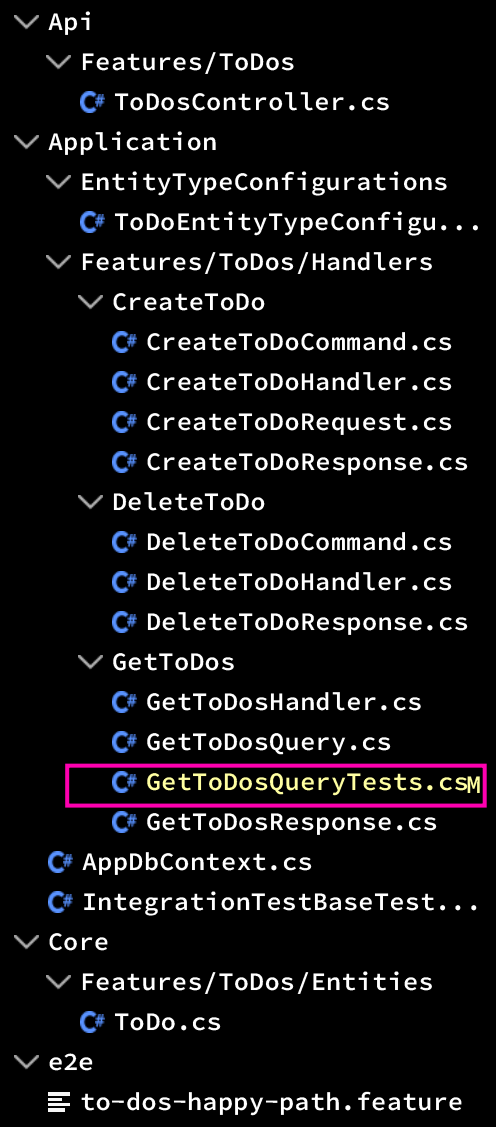
_dbTransaction = await _dbConnection.BeginTransactionAsync(
  System.Data.IsolationLevel.Serializable
);
```




```
public async Task DisposeAsync()
{
    ...
      try
      {
        await _dbTransaction.RollbackAsync();
      }
      finally
      {
        await _dbTransaction.DisposeAsync();
      }
    ...
}
```



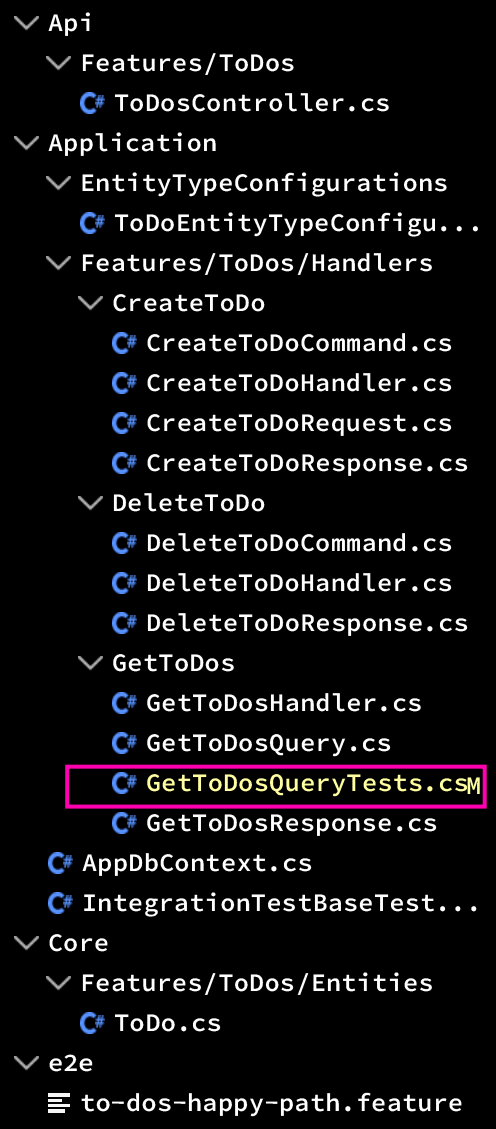

```
public class GetToDosQueryTests : IntegrationTestBase
{
  [Fact]
  public async Task GetAsync_ShouldOrderNewerToDosFirst()
  {
    var context = CreateAppDbContext();
```




```
var firstToDo = new ToDo { Name = "First" };
await AddEntityAndSaveAsync(context, firstToDo);

var secondToDo = new ToDo { Name = "Second" };
await AddEntityAndSaveAsync(context, secondToDo);

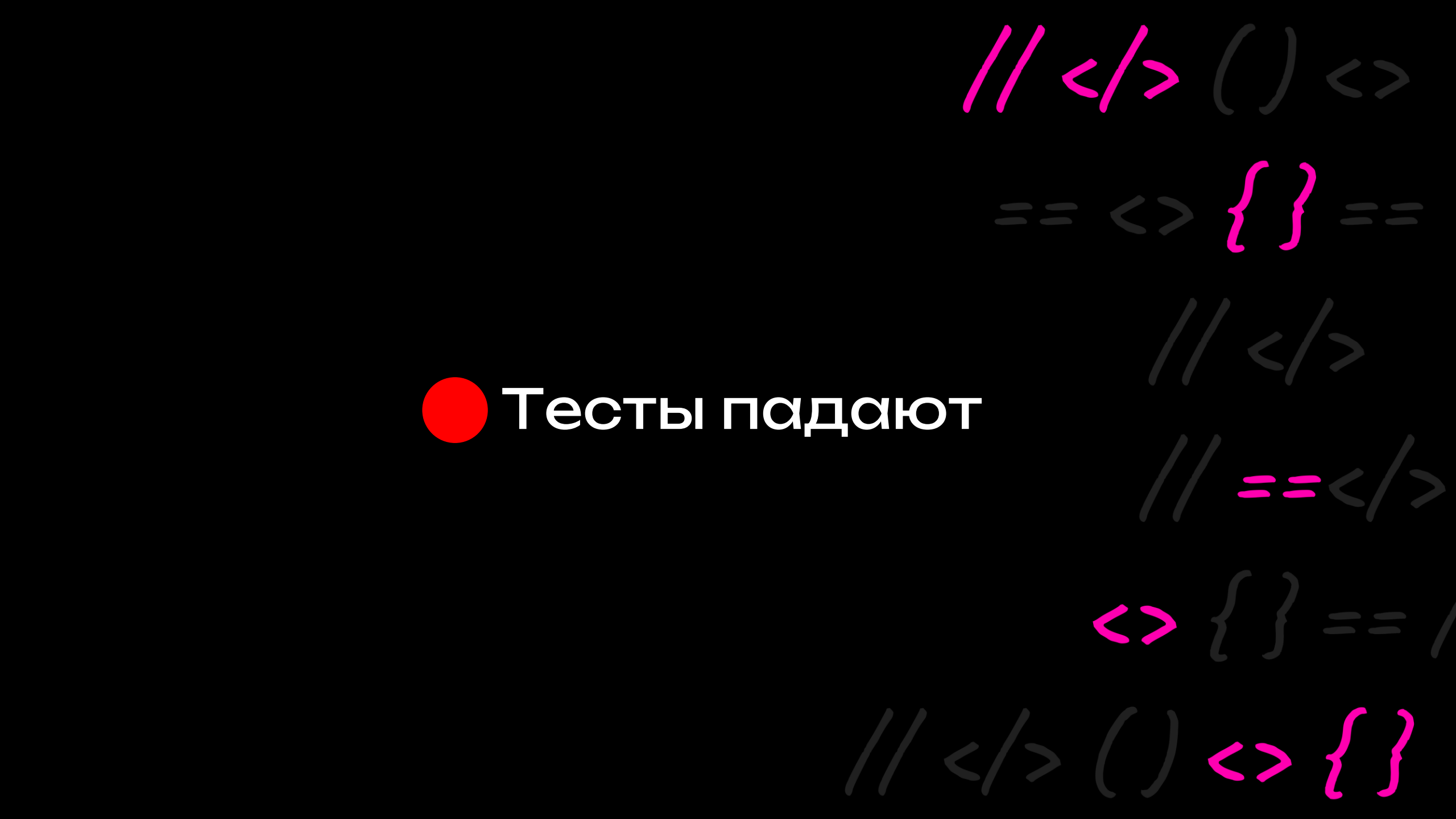
var thirdToDo = new ToDo { Name = "Third" };
await AddEntityAndSaveAsync(context, thirdToDo);
```

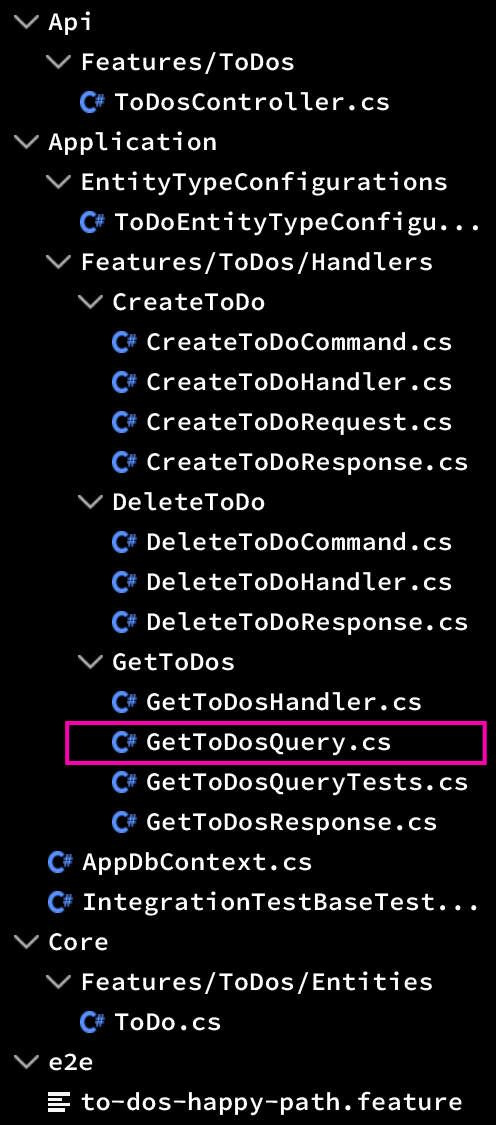



```
        var getToDosQuery = new GetToDosQuery(context);

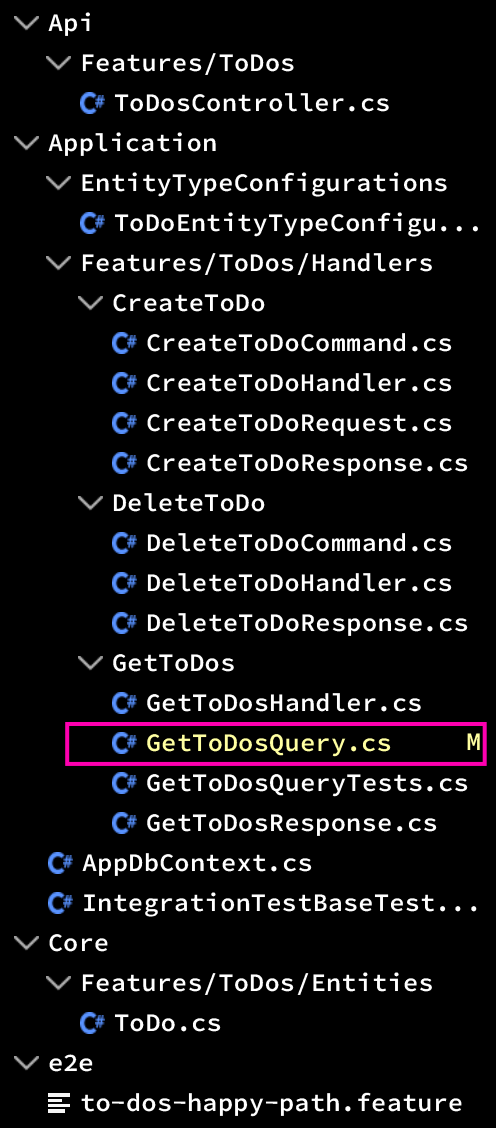
        var toDos = await getToDosQuery.GetAsync();

        Assert.Equal(thirdToDo.Name, toDos[0].Name);
        Assert.Equal(secondToDo.Name, toDos[1].Name);
        Assert.Equal(firstToDo.Name, toDos[2].Name);
    }
```

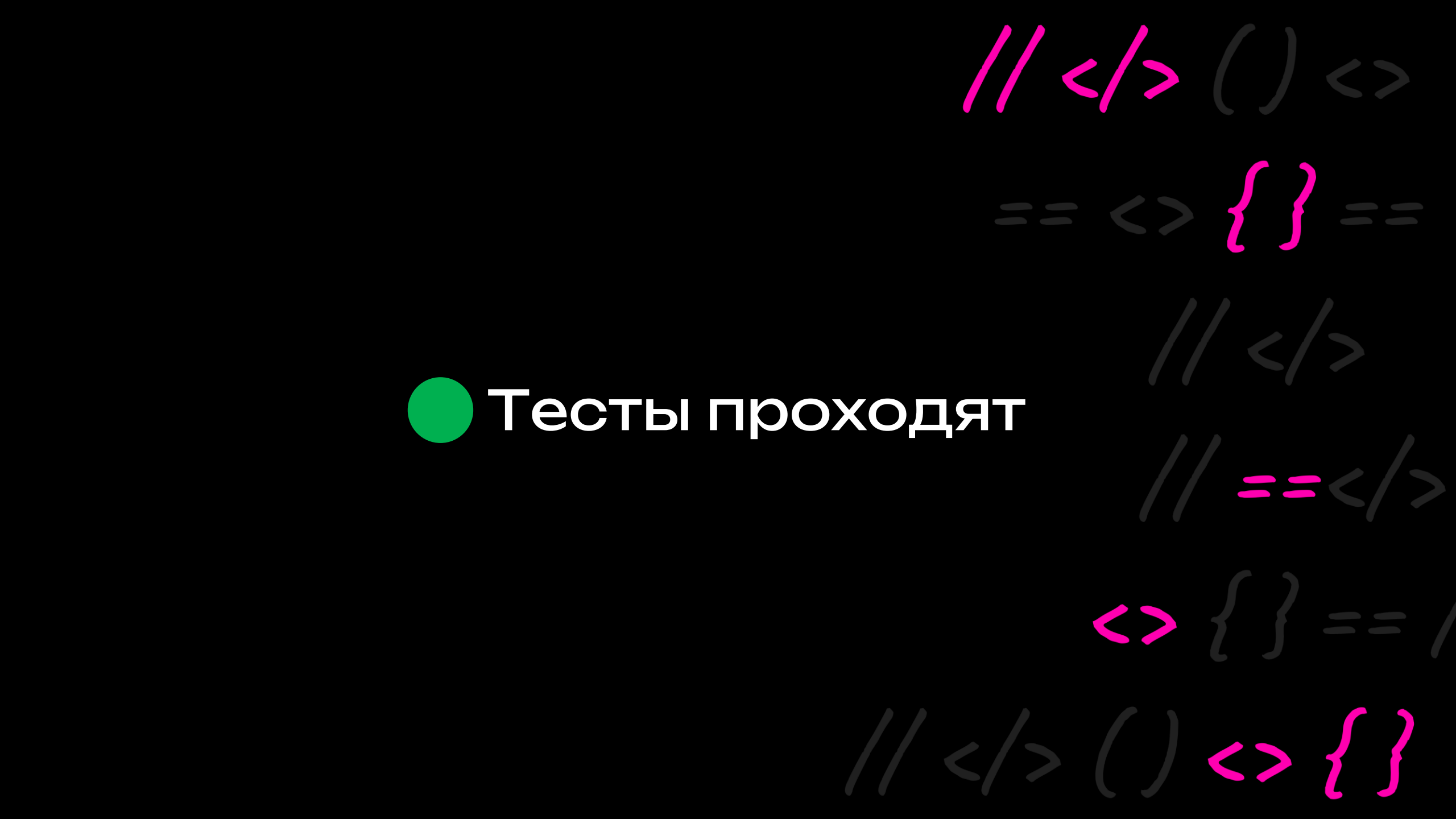

Тесты падают

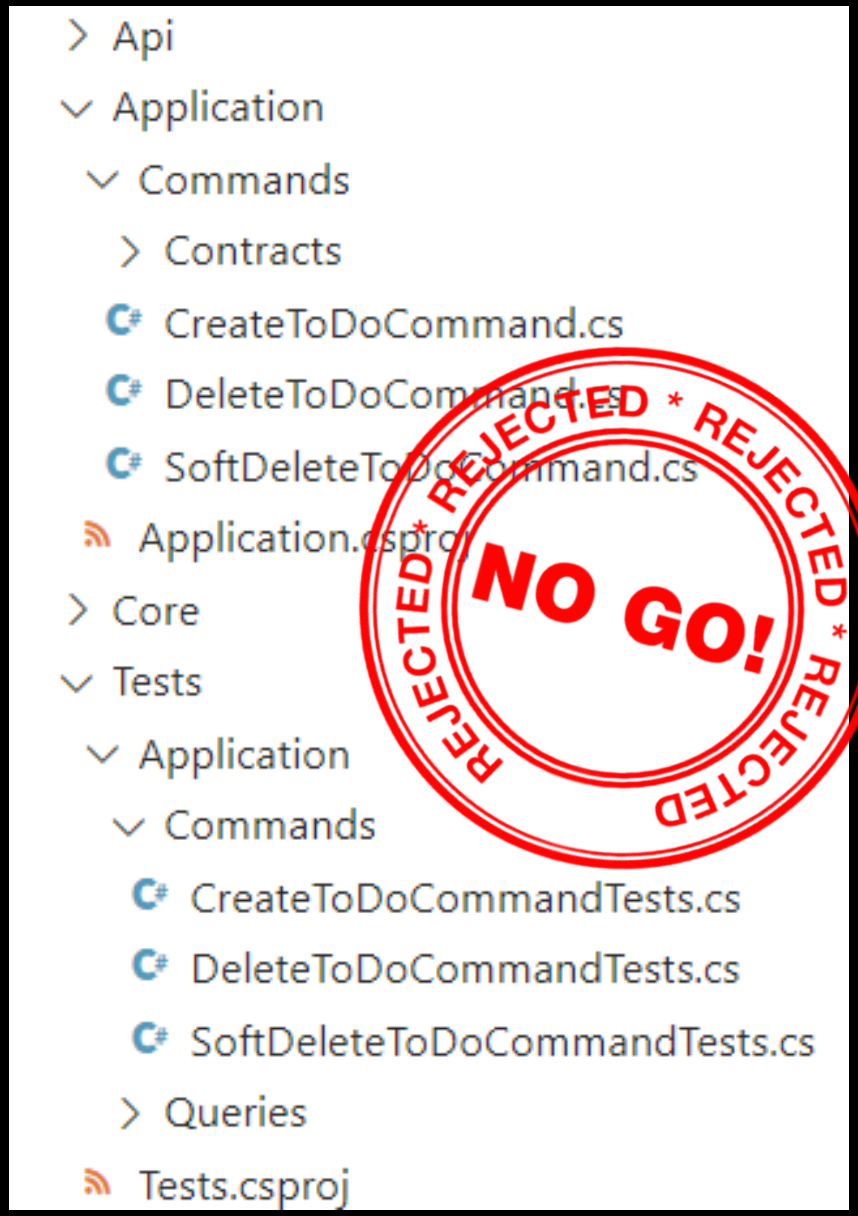



```
public class GetToDosQuery(AppDbContext context)
{
  public Task<List<ToDo>> GetAsync()
  {
    return context
      .QueryableAsNoTracking<ToDo>()
      .ToListAsync();
  }
}
```

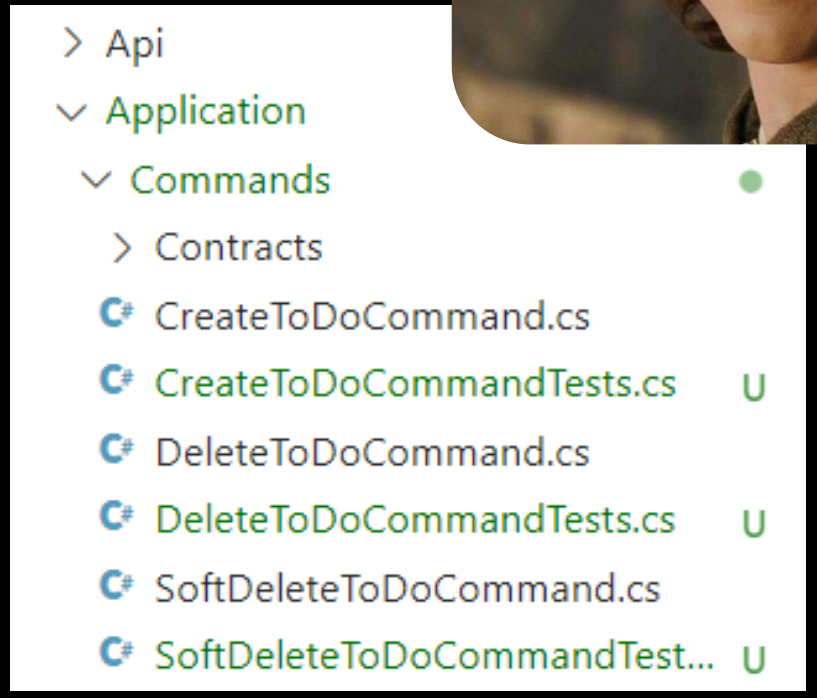



```
public class GetToDosQuery(AppDbContext context)
{
  public Task<List<ToDo>> GetAsync()
  {
    return context
      .QueryableAsNoTracking<ToDo>()
      .OrderByDescending(x => x.Id)
      .ToListAsync();
  }
}
```


Тесты проходят



JavaScript





Кадр из к/ф «Властелин колец: Возвращение короля» (The Lord of the Rings: The Return of the King), реж. Питер Джексон, 2003 г. Правообладатель: New Line Cinema / Warner Bros.

Статья на Хабре

Статус To Do

Хочу видеть статус To Do, зависящий от даты и времени создания

Чтобы отличать новые от старых и забытых и, соответственно, косвенно видеть важное и не важное

New — прошло меньше недели

Old — прошло от недели до четырёх

Forgotten — прошло больше четырёх недель



Эндпоинт на чтение со статусом

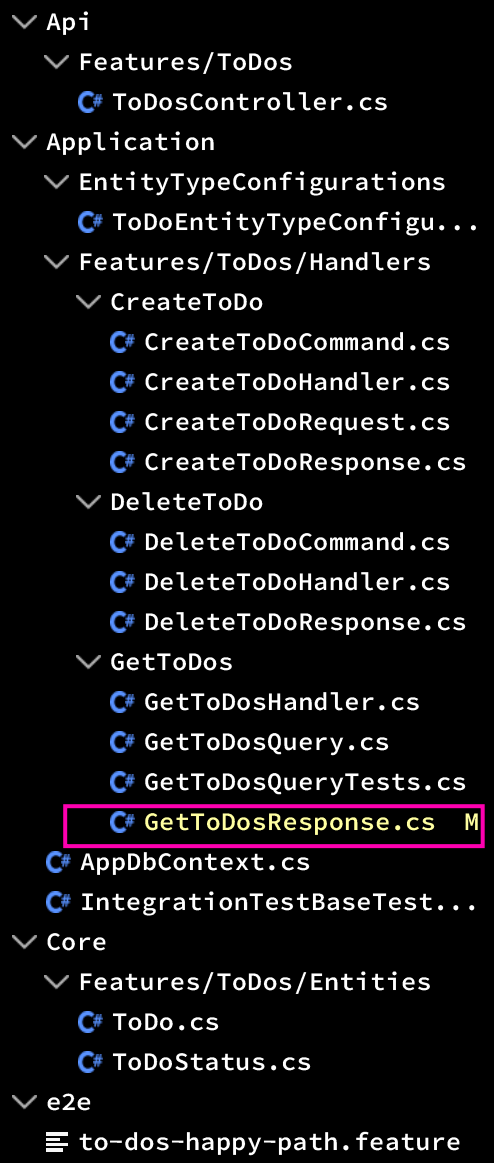
GET /to-dos

Response

```
{
  "toDos": [
    {
      "id": 1,
      "name": "Test",
      "status": "New" | "Old" | "Forgotten",
    }
  ]
}
```



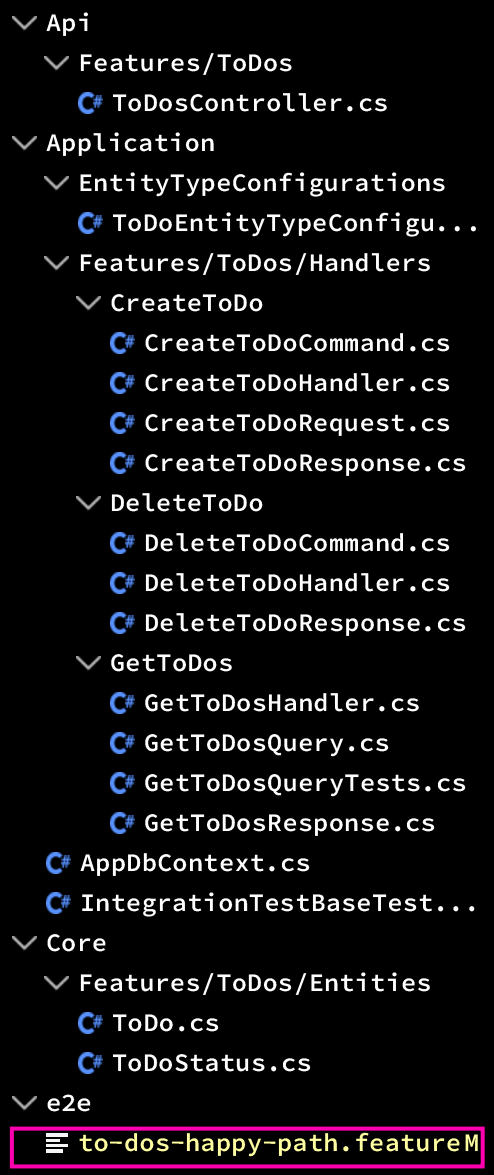

```
public enum ToDoStatus
{
    None = 0,
    New = 1,
    Old = 2,
    Forgotten = 3,
}
```



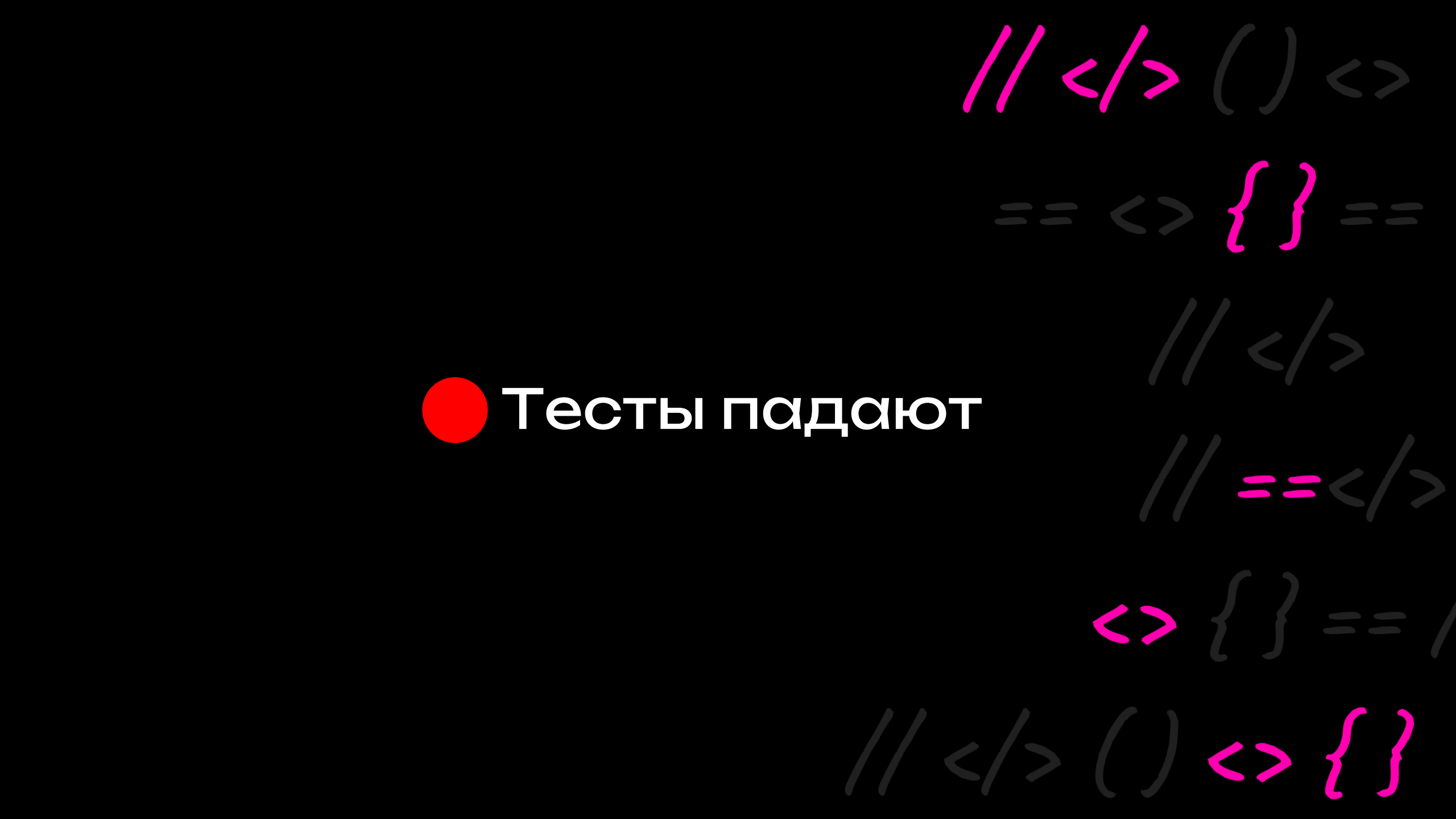

```
public class ToDoDto
{
    public required long Id { get; set; }

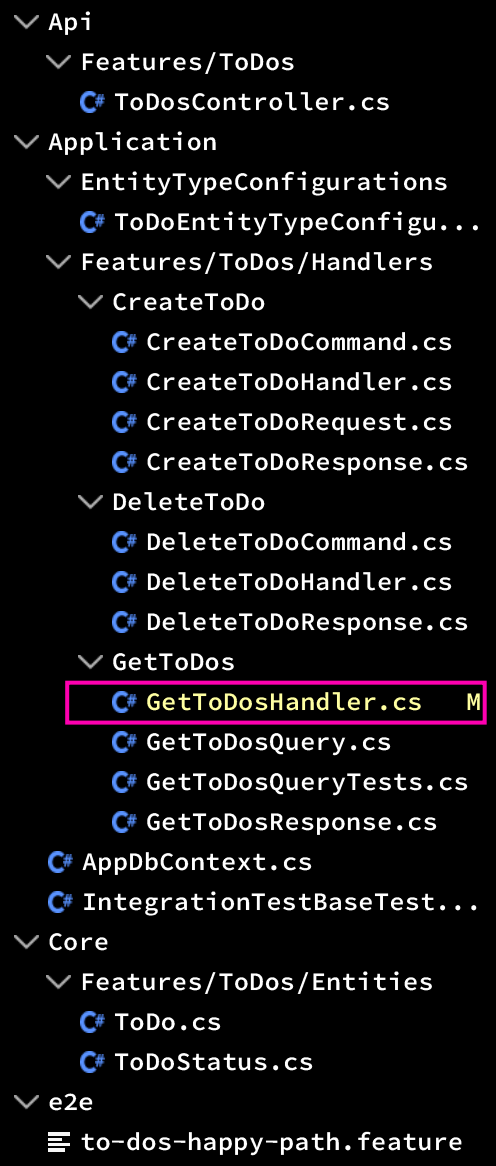
    public required string Name { get; set; }

    public required ToDoStatus Status { get; set; }
}
```

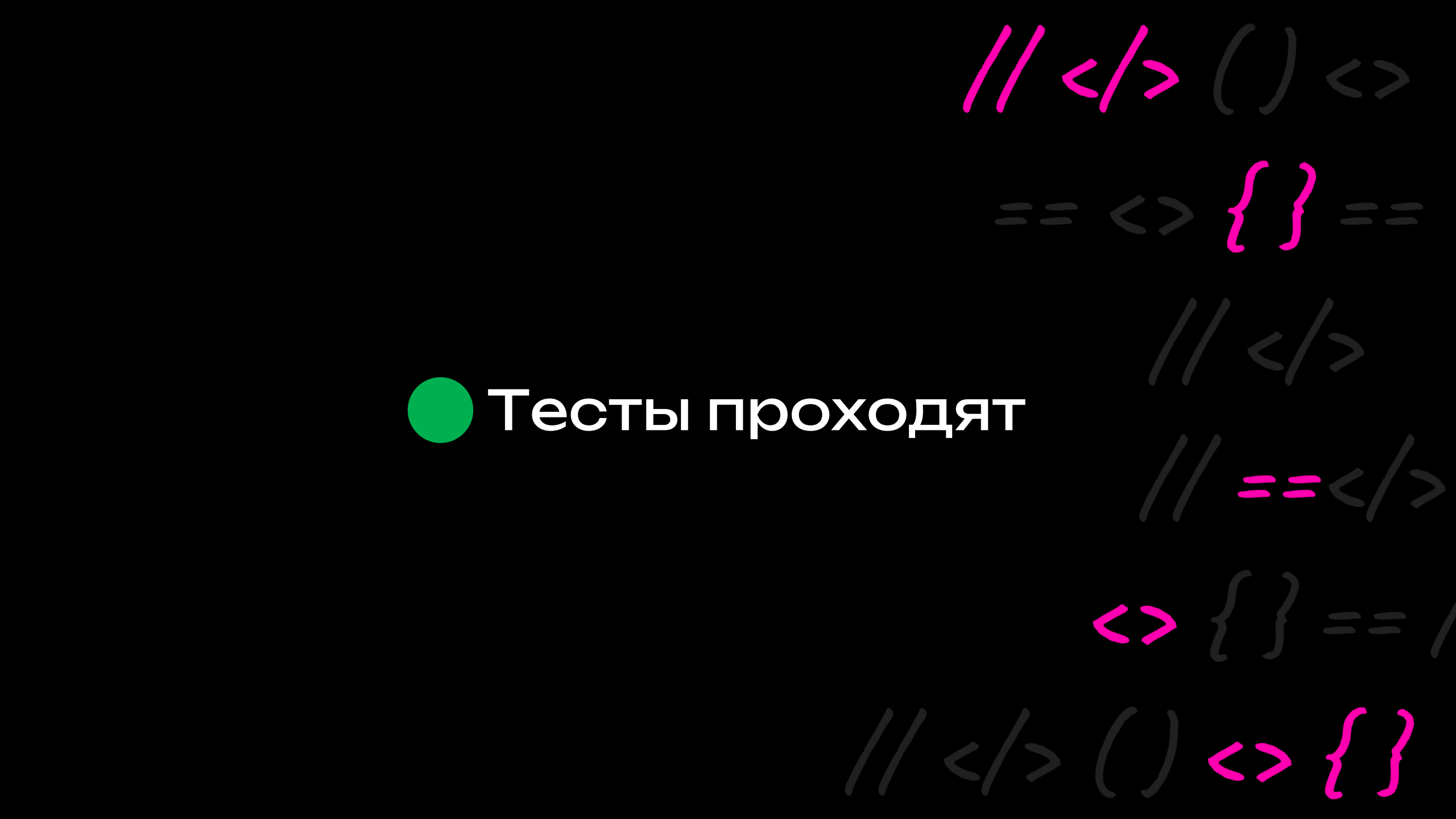


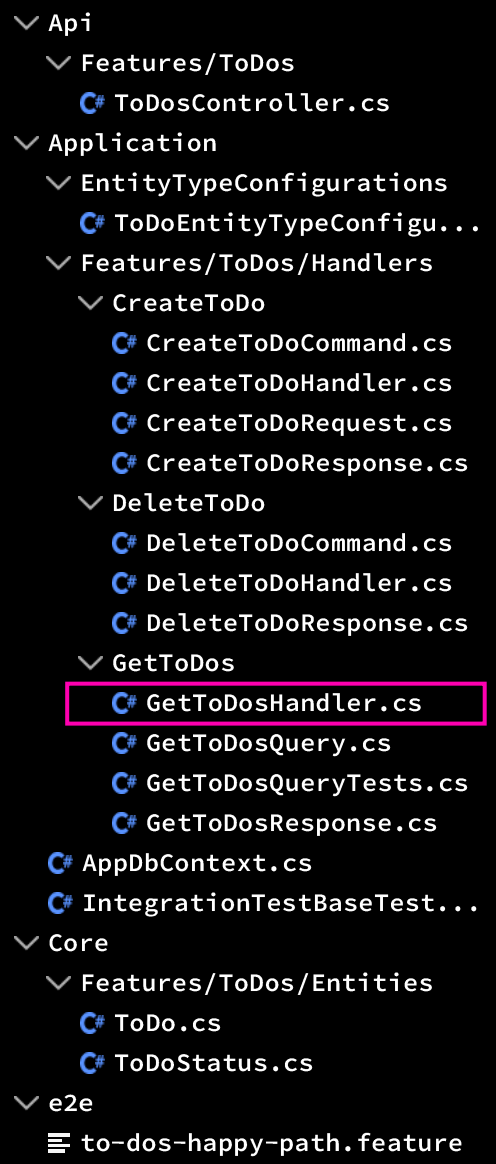

```
Given url apiRootUrl
And path 'to-dos'
When method GET
Then match response.toDos contains
"""
{
  "id": "#(newToDoId)",
  "name": "#(randomToDoName)",
  "status": "New",
}
"""
```


Тесты падают




```
return new GetToDosResponse
{
  ToDos = toDos
    .Select(x => new ToDoDto
    {
      Id = x.Id,
      Name = x.Name,
      Status = ToDoStatus.New,
    })
    .ToList(),
};
```

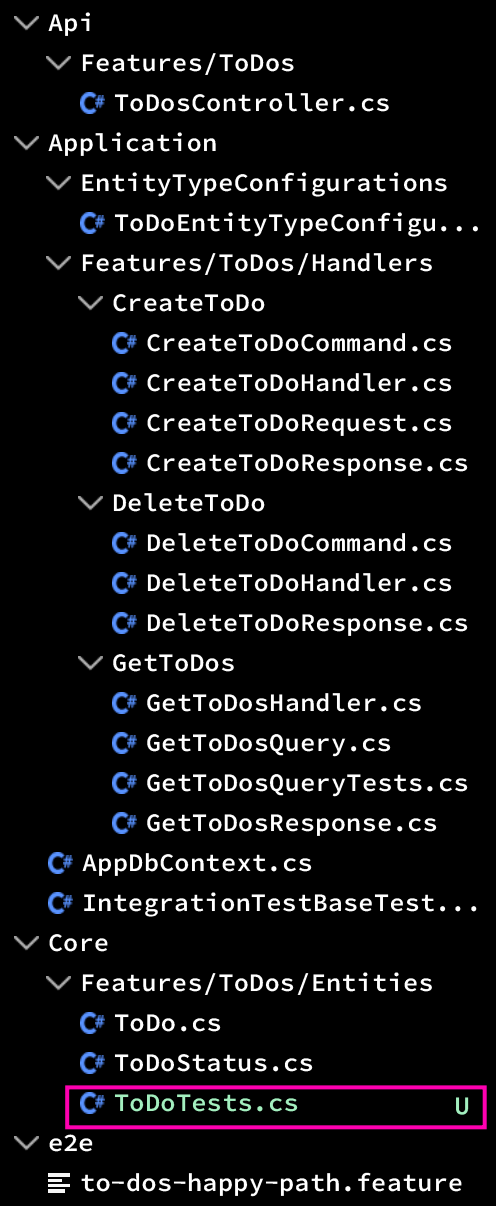

Тесты проходят




```
return new GetToDosResponse
{
  ToDos = toDos
    .Select(x => new ToDoDto
    {
      Id = x.Id,
      Name = x.Name,
      Status = ToDoStatus.New,
    })
    .ToList(),
};
```

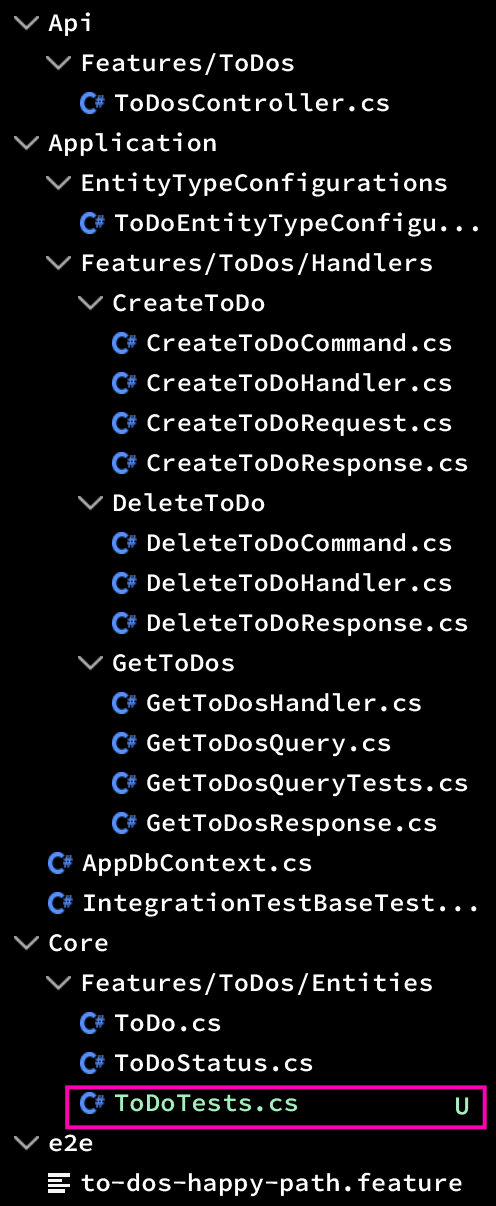
Куда положить логику с расчётом статуса?

Здесь идёт проектирование из-за TDD!




```
[Fact]
public void GetStatus_FiveDaysAgoIsNew()
{
  var dt = new DateTime(
    2026, 01, 02, 03, 04, 05, 678, DateTimeKind.Utc
  );

  var toDo = new ToDo {
    Name = "To Do",
    CreatedAtUtc = dt,
  };
```

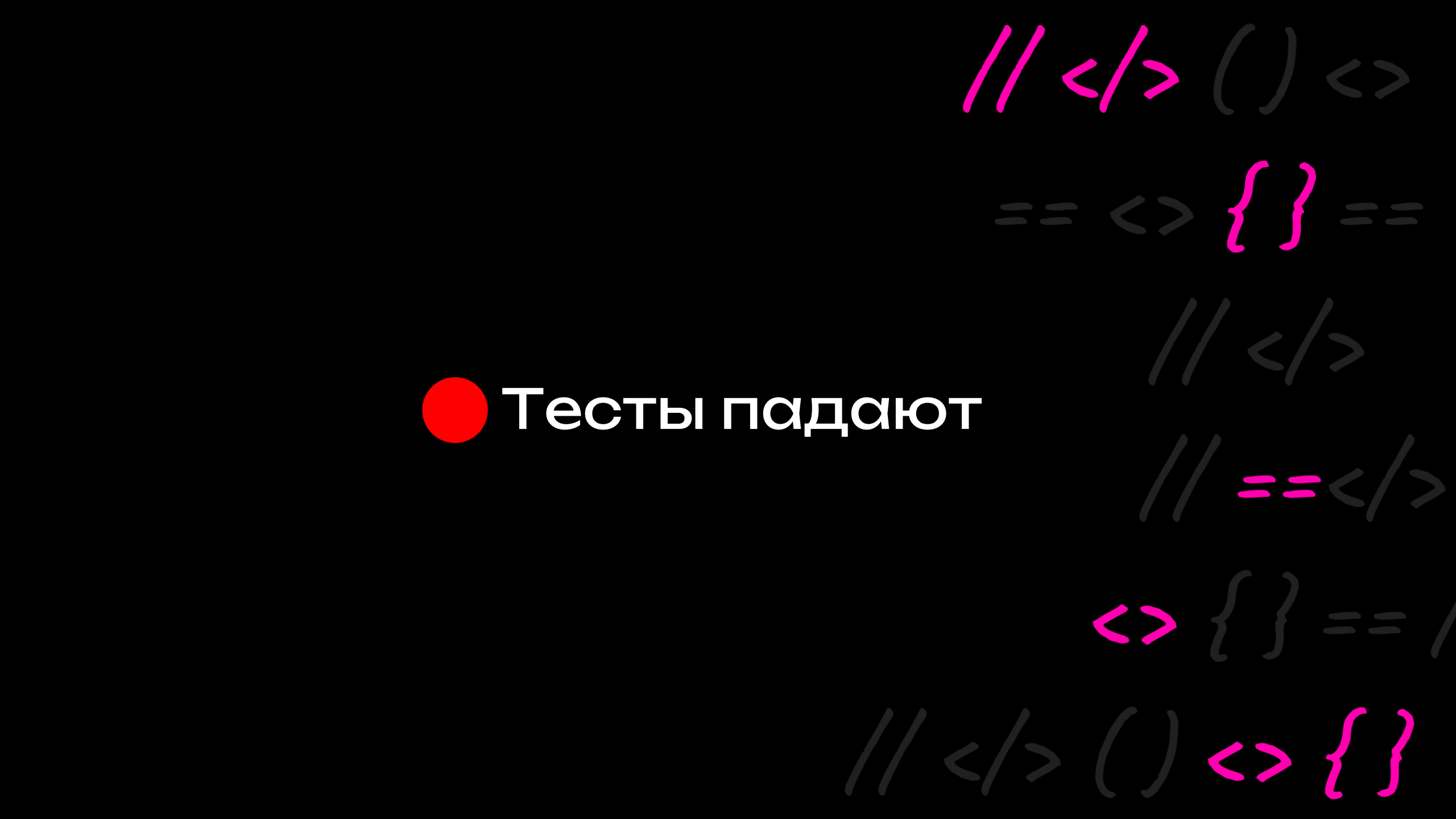



```
    var dateTimeProviderMock = new Mock<IDateTimeProvider>();

    dateTimeProviderMock.Setup(x => x.UtcNow)
        .Returns(dt.AddDays(5));

    var status = toDo.GetStatus(dateTimeProviderMock.Object);

    Assert.Equal(ToDoStatus.New, status);
}
```


Тесты падают




```
public interface IDateTimeProvider
{
  public DateTime UtcNow { get; }
}
```




```
public class ToDo
{
    public ToDo() { }

    public long Id { get; set; }

    public required string Name { get; set; }

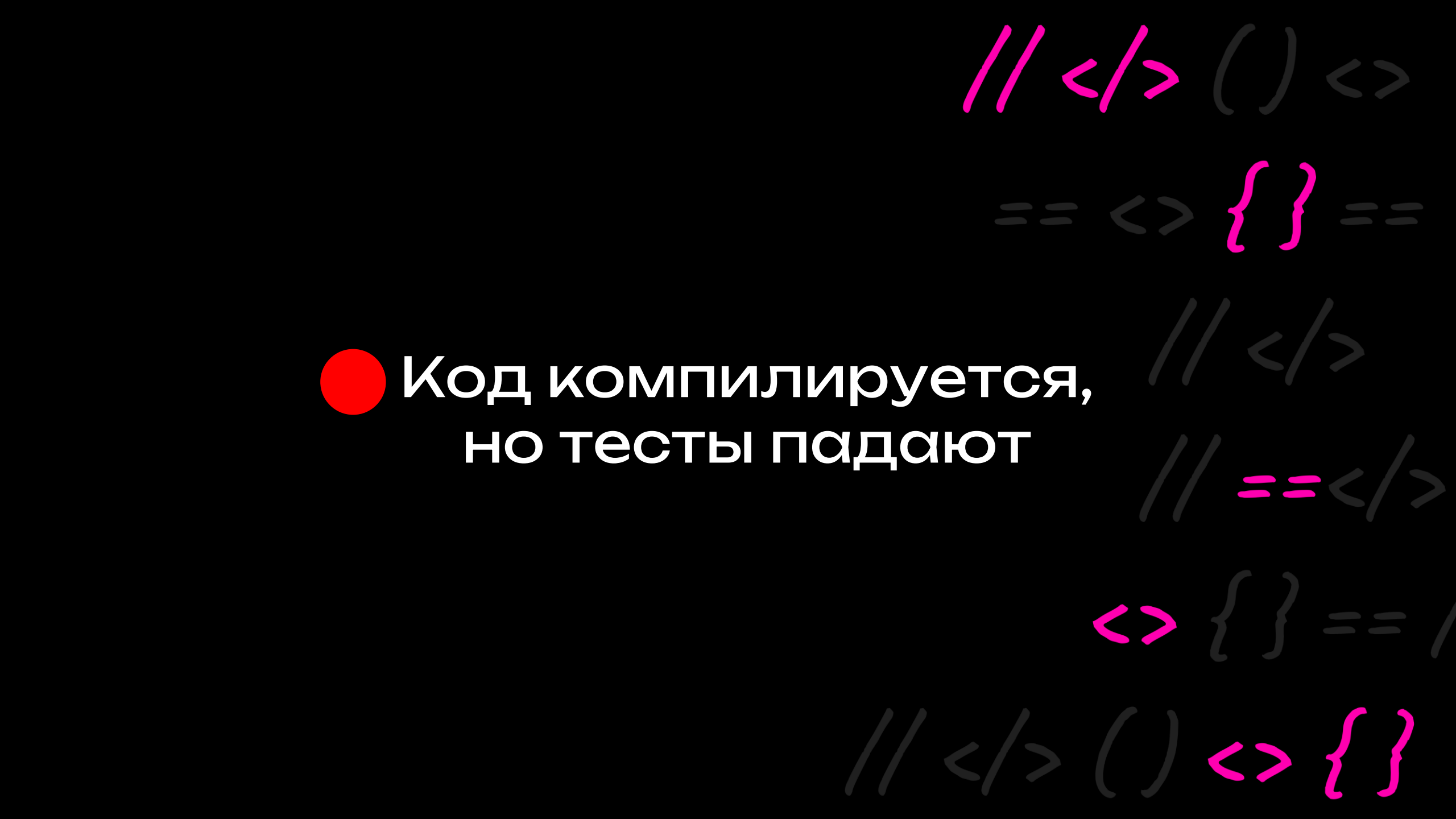
    public DateTime CreatedAtUtc { get; set; }
```




```
public ToDoStatus GetStatus(
  IDateTimeProvider dateTimeProvider
)
{
  var utcNow = dateTimeProvider.UtcNow;

  if (CreatedAtUtc > utcNow.AddDays(-7)
    && CreatedAtUtc < utcNow)
  {
    return ToDoStatus.New;
  }

  throw new InvalidOperationException(
    $"Not expected path of {nameof(GetStatus)}"
  );
}
```

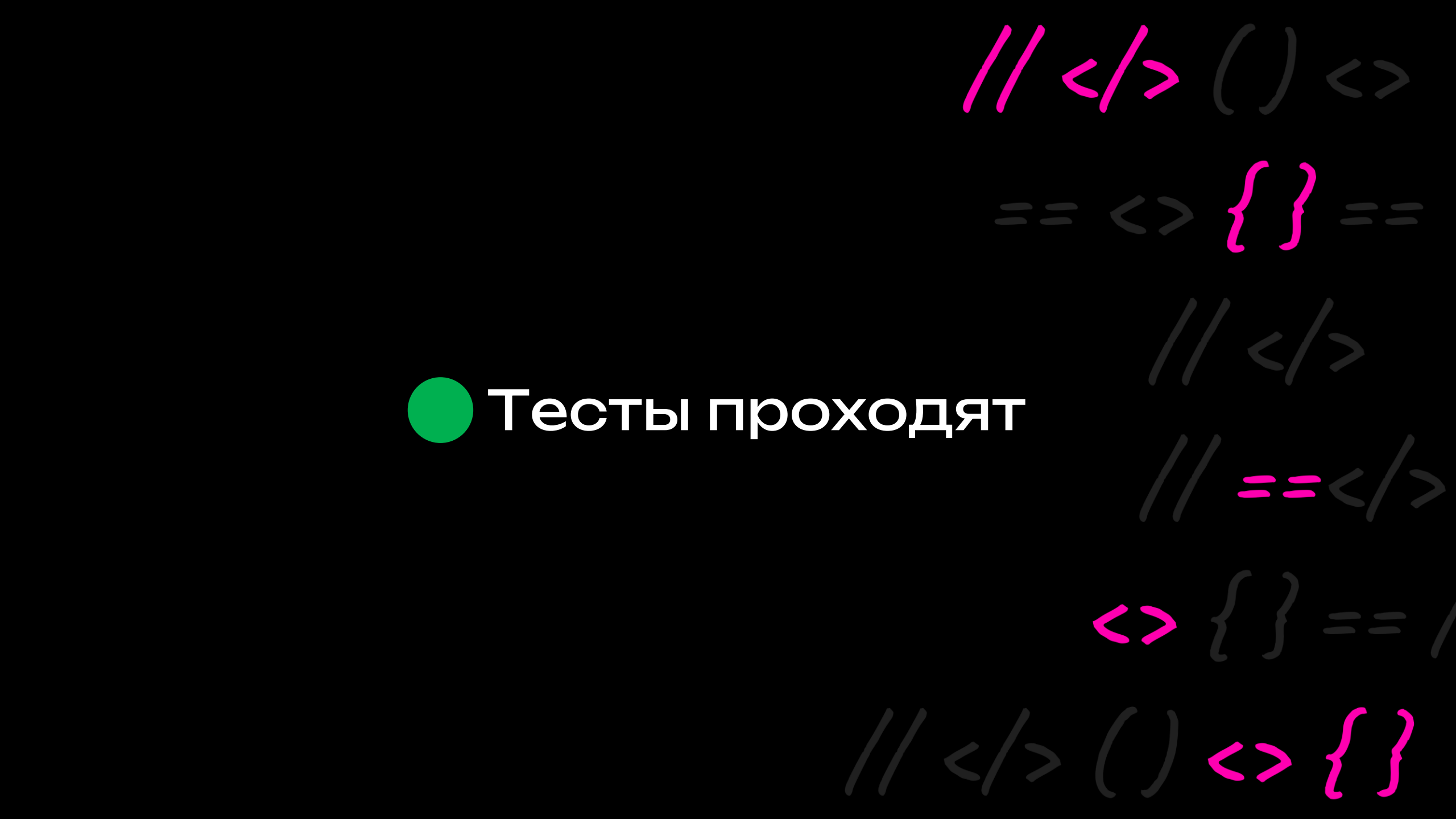

Код компилируется,
но тесты падают

GetAsync_ShouldOrderNewerToDosFirst

FAILED: GetAsync_ShouldOrderNewerToDosFirst (141ms)
Microsoft.EntityFrameworkCore.DbUpdateException : An error occurred while saving the entity changes. See the inner exception for details. ---- Npgsql.PostgresException : 42703: column "created_at_utc" of relation "to_dos" does not exist

Создаём миграцию для CreatedAtUtc колонки

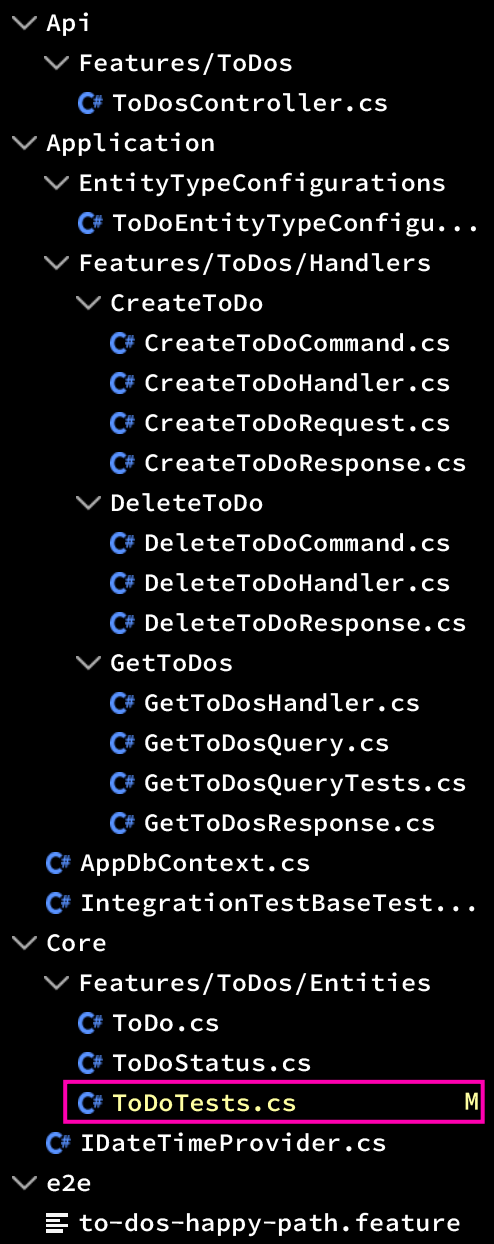
```
dotnet ef migrations add AddToDosCreatedAtUtc
  --startup-project ./Api/Api.csproj
  --project ./Application/Application.csproj
  --context AppDbContext
  --verbose
```


Тесты проходят




```
[Fact]
public void GetStatus_EightDaysAgoIsOld()
{
  var dt = new DateTime(
    2026, 01, 02, 03, 04, 05, 678, DateTimeKind.Utc
  );

  var toDo = new ToDo {
    Name = "To Do",
    CreatedAtUtc = dt,
  };
```

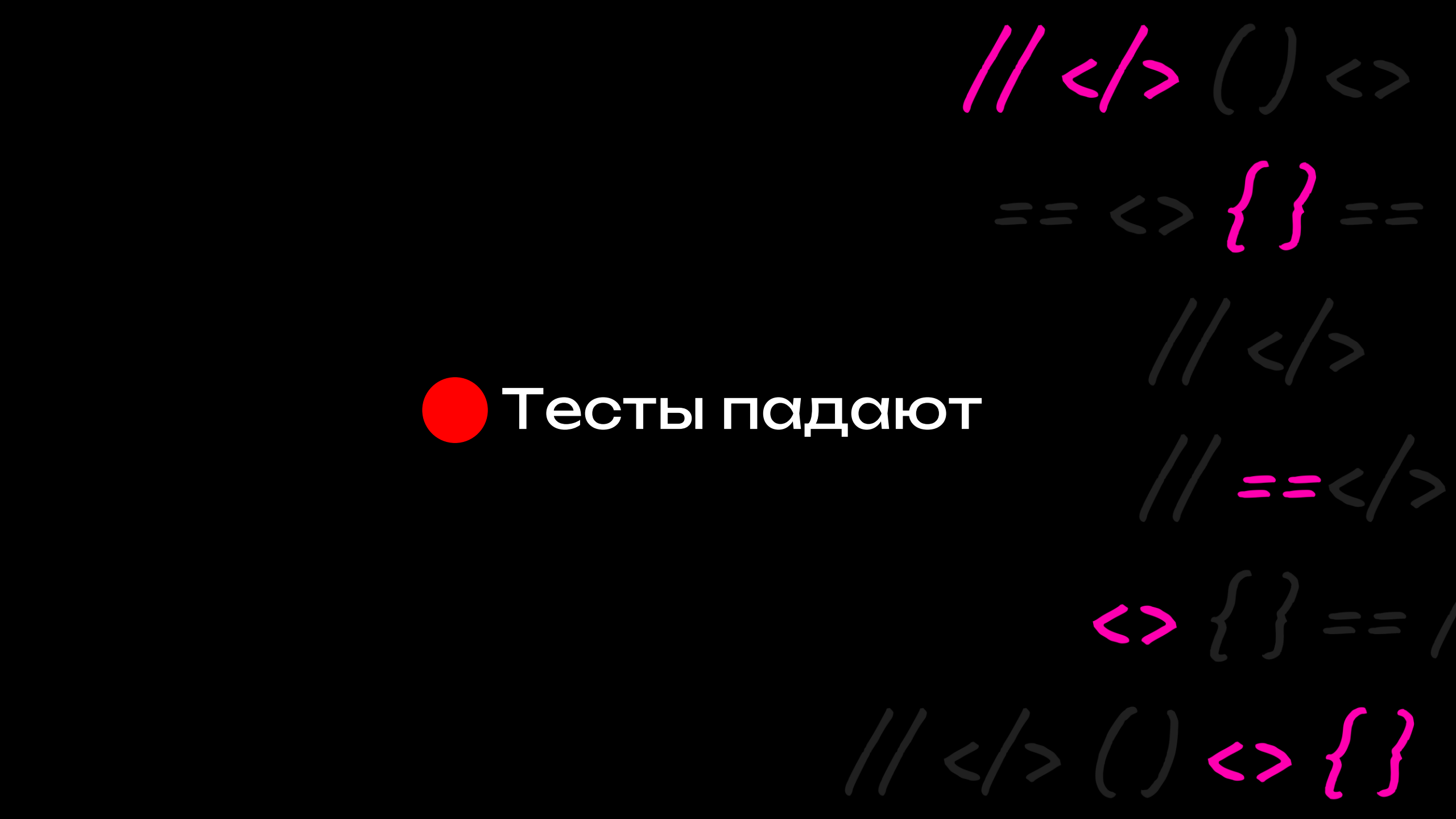



```
    var dateTimeProviderMock = new Mock<IDateTimeProvider>();

    dateTimeProviderMock.Setup(x => x.UtcNow)
      .Returns(dt.AddDays(8));

    var status = toDo.GetStatus(dateTimeProviderMock.Object);

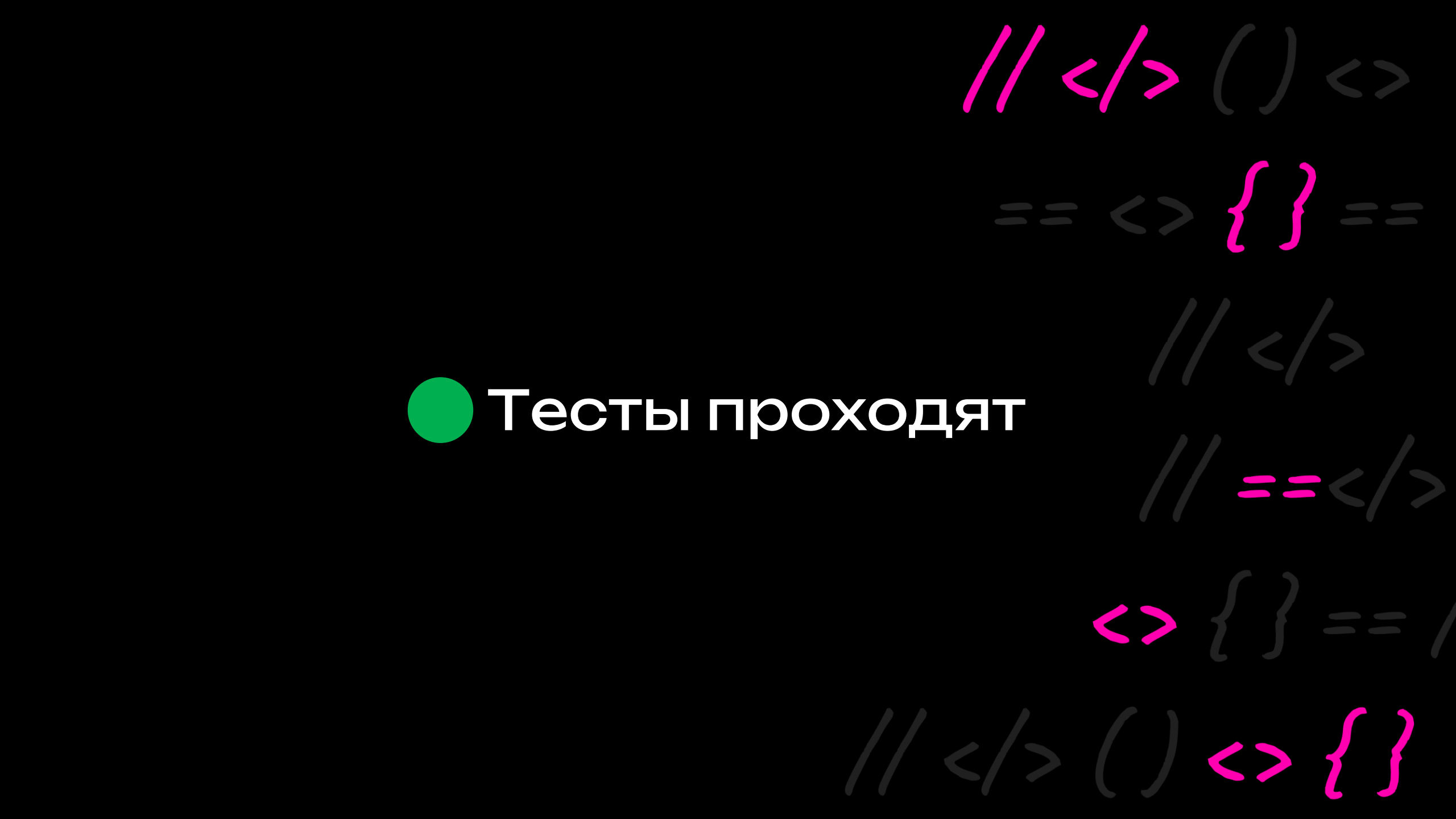
    Assert.Equal(ToDoStatus.Old, status);
}
```


Тесты падают



GetStatus

```
if (CreatedAtUtc > utcNow.AddDays(-7)
  && CreatedAtUtc < utcNow)
{
  return ToDoStatus.New;
}
else if (CreatedAtUtc > utcNow.AddDays(-28)
  && CreatedAtUtc <= utcNow.AddDays(-7))
{
  return ToDoStatus.Old;
}
```


Тесты проходят

```
public async Task GetStatus_FiveDaysAgoIsNew()
{
  var dt = new DateTime(2026, 01, 02, 03, 04, 05, 678, DateTimeKind.Utc);

  var toDo = new Todo { Name = "To Do", CreatedAtUtc = dt };

  var dateTimeProviderMock = new Mock<IDateTimeProvider>();

  dateTimeProviderMock.Setup(x => x.UtcNow).Returns(dt.AddDays(5));

  var status = toDo.GetStatus(dateTimeProviderMock.Object);

  Assert.Equal(ToDoStatus.New, status);
}
```

```
public async Task GetStatus_EightDaysAgoIsOld()
{
  var dt = new DateTime(2026, 01, 02, 03, 04, 05, 678, DateTimeKind.Utc);

  var toDo = new Todo { Name = "To Do", CreatedAtUtc = dt };

  var dateTimeProviderMock = new Mock<IDateTimeProvider>();

  dateTimeProviderMock.Setup(x => x.UtcNow).Returns(dt.AddDays(8));

  var status = toDo.GetStatus(dateTimeProviderMock.Object);

  Assert.Equal(ToDoStatus.Old, status);
}
```

```
public async Task GetStatus_FiveDaysAgoIsNew()
public async Task GetStatus_EightDaysAgoIsOld()
{
    var dt = new DateTime(2026, 01, 02, 03, 04, 05, 678, DateTimeKind.Utc);

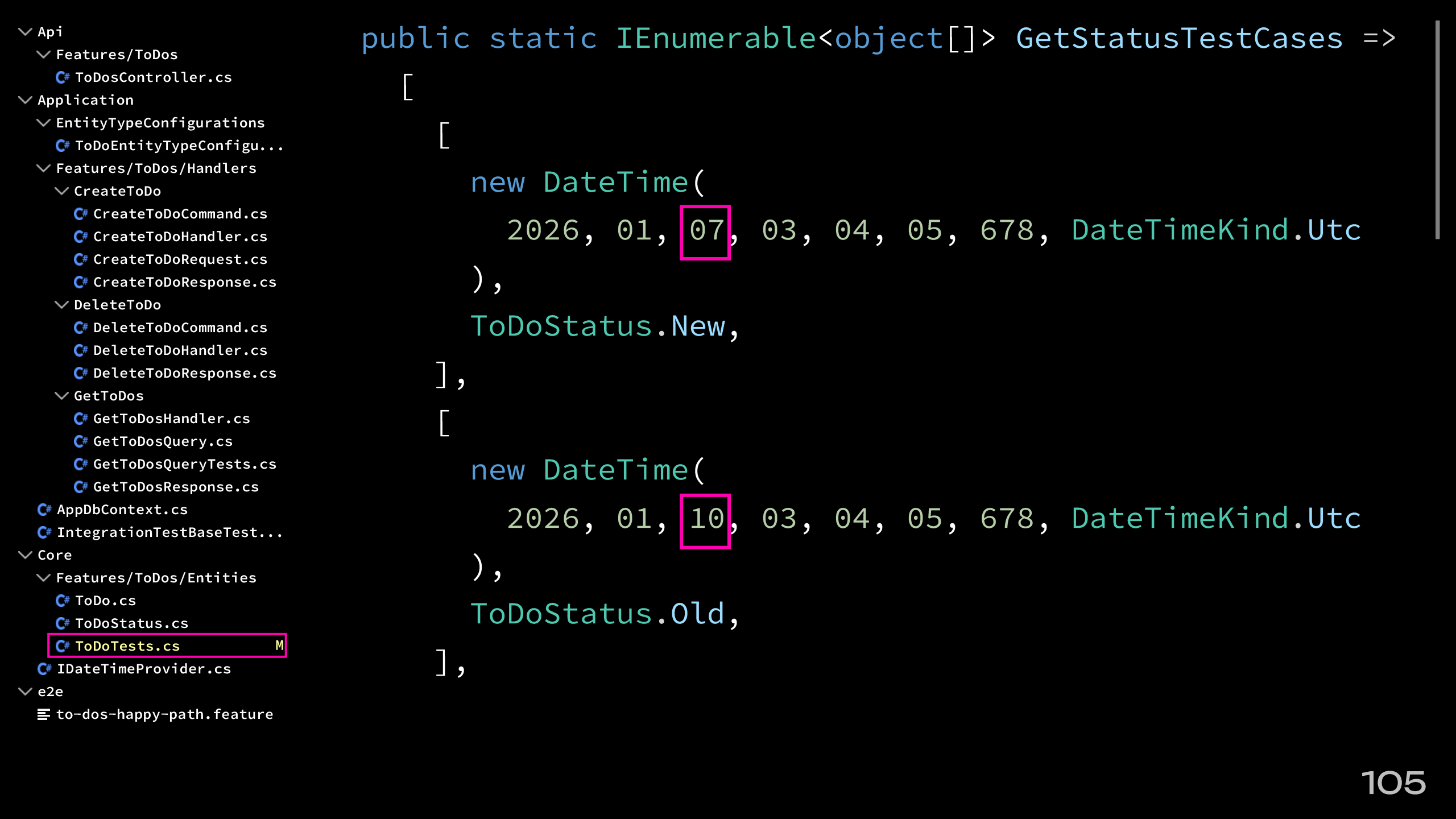
    var toDo = new Todo { Name = "To Do", CreatedAtUtc = dt };

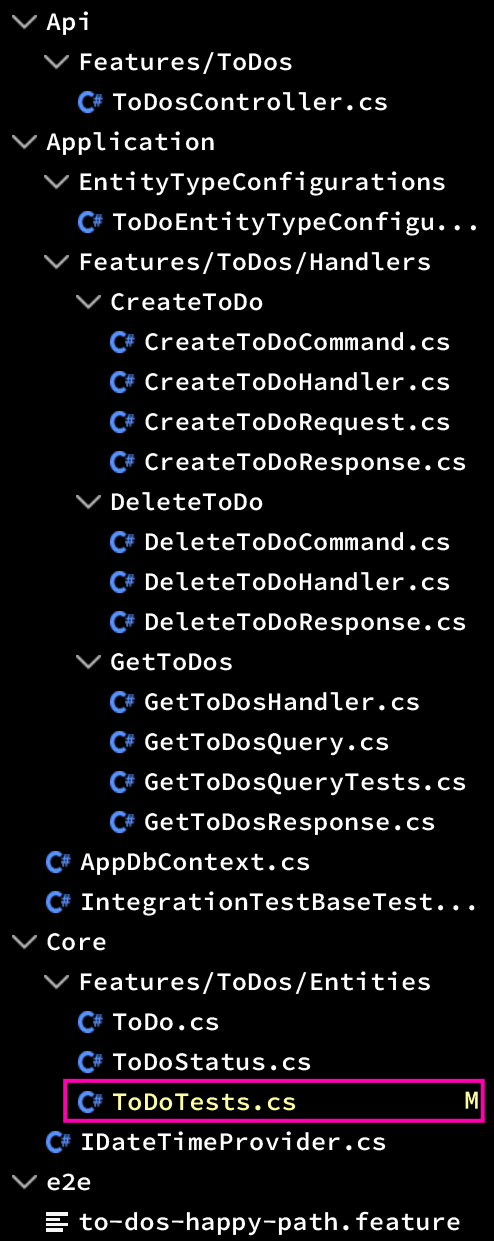
    var dateTimeProviderMock = new Mock<IDateTimeProvider>();

    dateTimeProviderMock.Setup(x => x.UtcNow).Returns(dt.AddDays(5));
    dateTimeProviderMock.Setup(x => x.UtcNow).Returns(dt.AddDays(8));

    var status = toDo.GetStatus(dateTimeProviderMock.Object);

    Assert.Equal(ToDoStatus.New, status);
    Assert.Equal(ToDoStatus.Old, status);
}
```


Api
Features/ToDos
ToDosController.cs
Application
EntityTypeConfigurations
ToDoEntityTypeConfigu...
Features/ToDos/Handlers
CreateToDo
CreateToDoCommand.cs
CreateToDoHandler.cs
CreateToDoRequest.cs
CreateToDoResponse.cs
DeleteToDo
DeleteToDoCommand.cs
DeleteToDoHandler.cs
DeleteToDoResponse.cs
GetToDos
GetToDosHandler.cs
GetToDosQuery.cs
GetToDosQueryTests.cs
GetToDosResponse.cs
AppDbContext.cs
IntegrationTestBaseTest...
Core
Features/ToDos/Entities
ToDo.cs
ToDoStatus.cs
ToDoTests.cs M
IDateTimeProvider.cs
e2e
to-dos-happy-path.feature
public static IEnumerable<object[]> GetStatusTestCases =>
[
[
new DateTime(
2026, 01, 07, 03, 04, 05, 678, DateTimeKind.Utc
),
ToDoStatus.New,
],
[
new DateTime(
2026, 01, 10, 03, 04, 05, 678, DateTimeKind.Utc
),
ToDoStatus.Old,
],

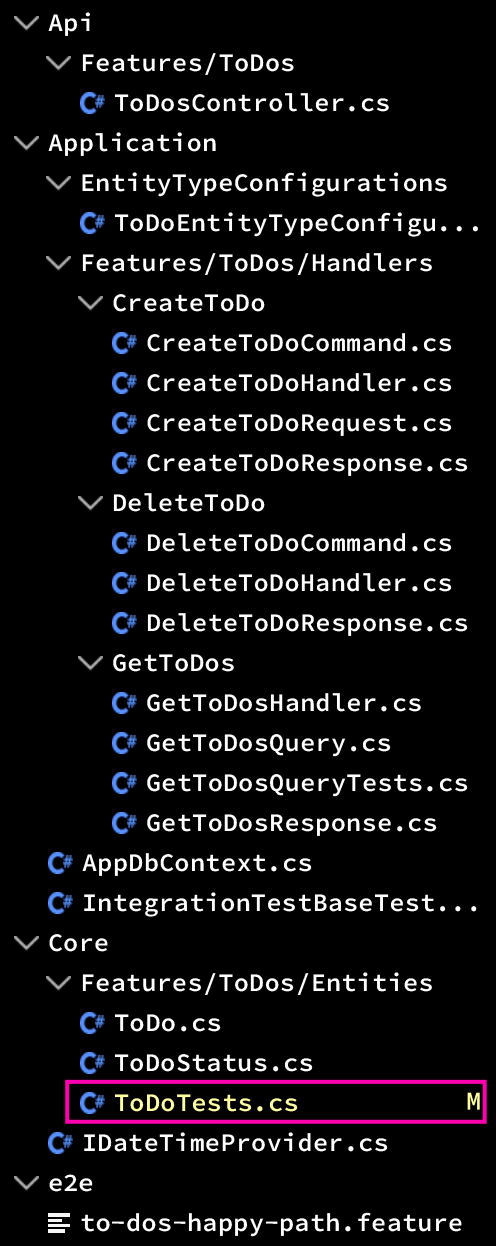



```
    [
      new DateTime(
        2026, 01, 30, 03, 04, 05, 678, DateTimeKind.Utc
      ),
      ToDoStatus.Forgotten,
    ],
  ];
```




```
[Theory]
[MemberData(nameof(GetStatusTestCases))]
public void GetStatus(
  DateTime utcNow,
  ToDoStatus expectedStatus
)
{
  var dt = new DateTime(
    2026, 01, 02, 03, 04, 05, 678, DateTimeKind.Utc
  );

  var toDo = new ToDo {
    Name = "To Do",
    CreatedAtUtc = dt,
  };
```



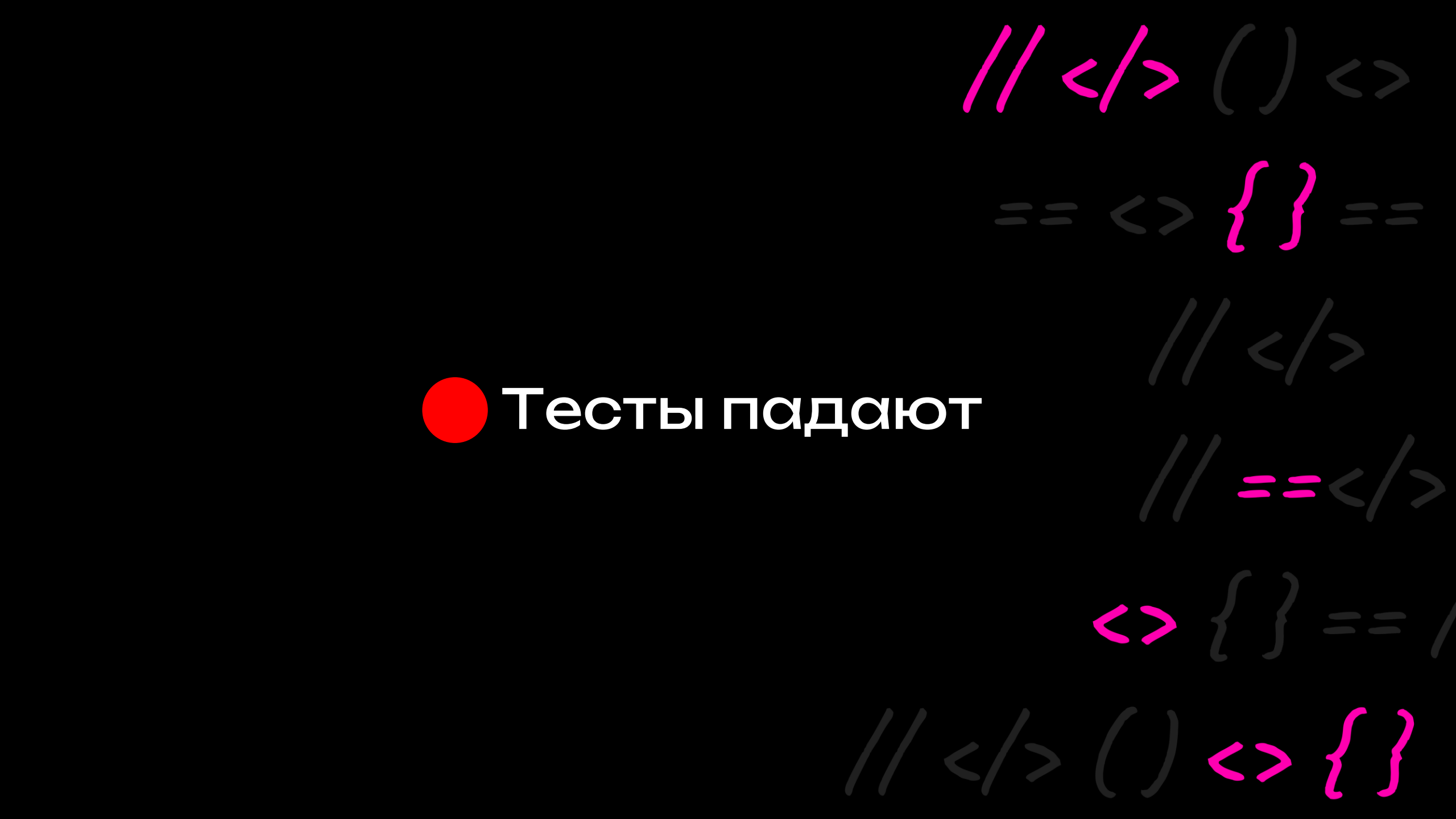

```
    var dateTimeProviderMock = new Mock<IDateTimeProvider>();

    dateTimeProviderMock.Setup(x => x.UtcNow)
        .Returns(utcNow);

    var status = toDo.GetStatus(dateTimeProviderMock.Object);

    Assert.Equal(expectedStatus, status);
}
```

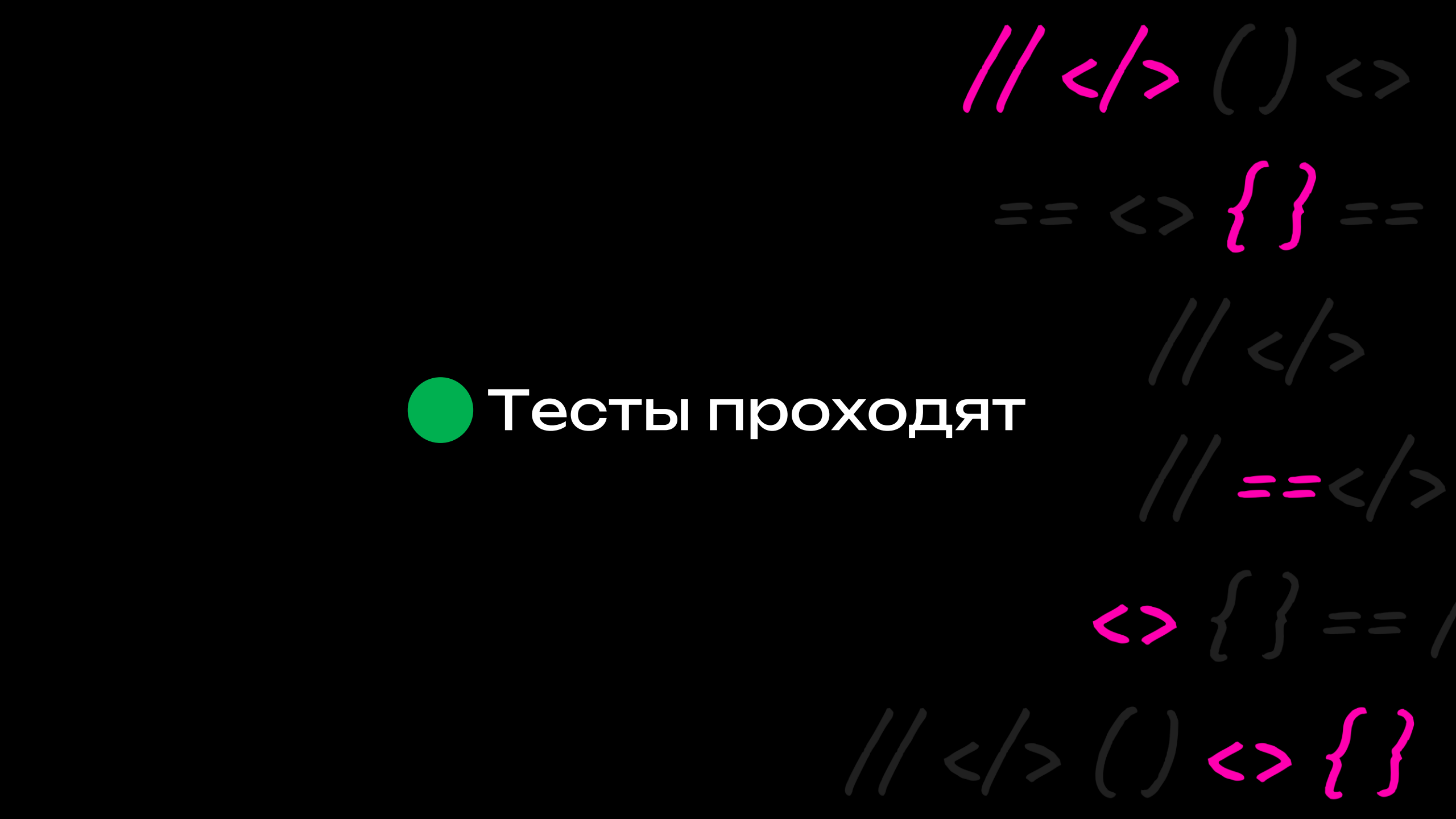
Статья про эти тихники


Тесты падают



GetStatus

```
if (CreatedAtUtc > utcNow.AddDays(-7)
  && CreatedAtUtc < utcNow)
{
  return ToDoStatus.New;
}
else if (CreatedAtUtc > utcNow.AddDays(-28)
  && CreatedAtUtc <= utcNow.AddDays(-7))
{
  return ToDoStatus.Old;
}
else if (CreatedAtUtc <= utcNow.AddDays(-28))
{
  return ToDoStatus.Forgotten;
}
```


Тесты проходят

- Api
  - Features/ToDos
    - ToDosController.cs
- Application
  - EntityTypeConfigurations
    - ToDoEntityTypeConfigu...
  - Features/ToDos/Handlers
    - CreateToDo
      - CreateToDoCommand.cs
      - CreateToDoHandler.cs
      - CreateToDoRequest.cs
      - CreateToDoResponse.cs
    - DeleteToDo
      - DeleteToDoCommand.cs
      - DeleteToDoHandler.cs
      - DeleteToDoResponse.cs
    - GetToDos
      - GetToDosHandler.cs
      - GetToDosQuery.cs
      - GetToDosQueryTests.cs
      - GetToDosResponse.cs
  - AppDbContext.cs
  - IntegrationTestBaseTest...
- Core
  - Features/ToDos/Entities
    - ToDo.cs
    - ToDoStatus.cs
    - ToDoTests.cs
  - IDateTimeProvider.cs
- e2e
  - to-dos-happy-path.feature

GetStatus

```
if (CreatedAtUtc > utcNow.AddDays(-7)
  && CreatedAtUtc < utcNow)
{
  return ToDoStatus.New;
}
else if (CreatedAtUtc > utcNow.AddDays(-28)
  && CreatedAtUtc <= utcNow.AddDays(-7))
{
  return ToDoStatus.Old;
}
else if (CreatedAtUtc <= utcNow.AddDays(-28))
{
  return ToDoStatus.Forgotten;
}
```



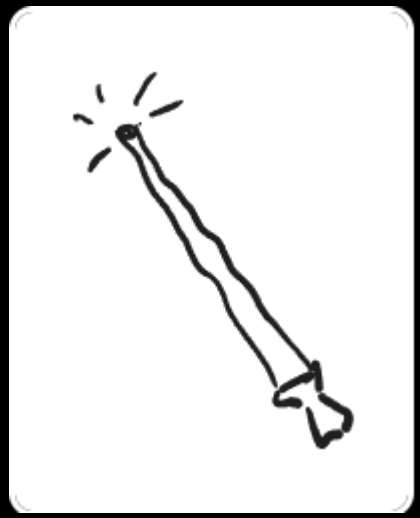
```
public ToDoStatus GetStatus(
  IDateTimeProvider dateTimeProvider
) {
  const int newForDays = 7;
  const int forgottenAfterDays = 28;

  var ageInDays = (dateTimeProvider.UtcNow - CreatedAtUtc)
    .TotalDays;

  if (ageInDays < newForDays)
    return ToDoStatus.New;

  if (ageInDays < forgottenAfterDays)
    return ToDoStatus.Old;

  return ToDoStatus.Forgotten;
}
```



- Тесты всё ещё проходят

Техники тест-дизайна



Шеф, почему тестов 3?

А сколько нужно?

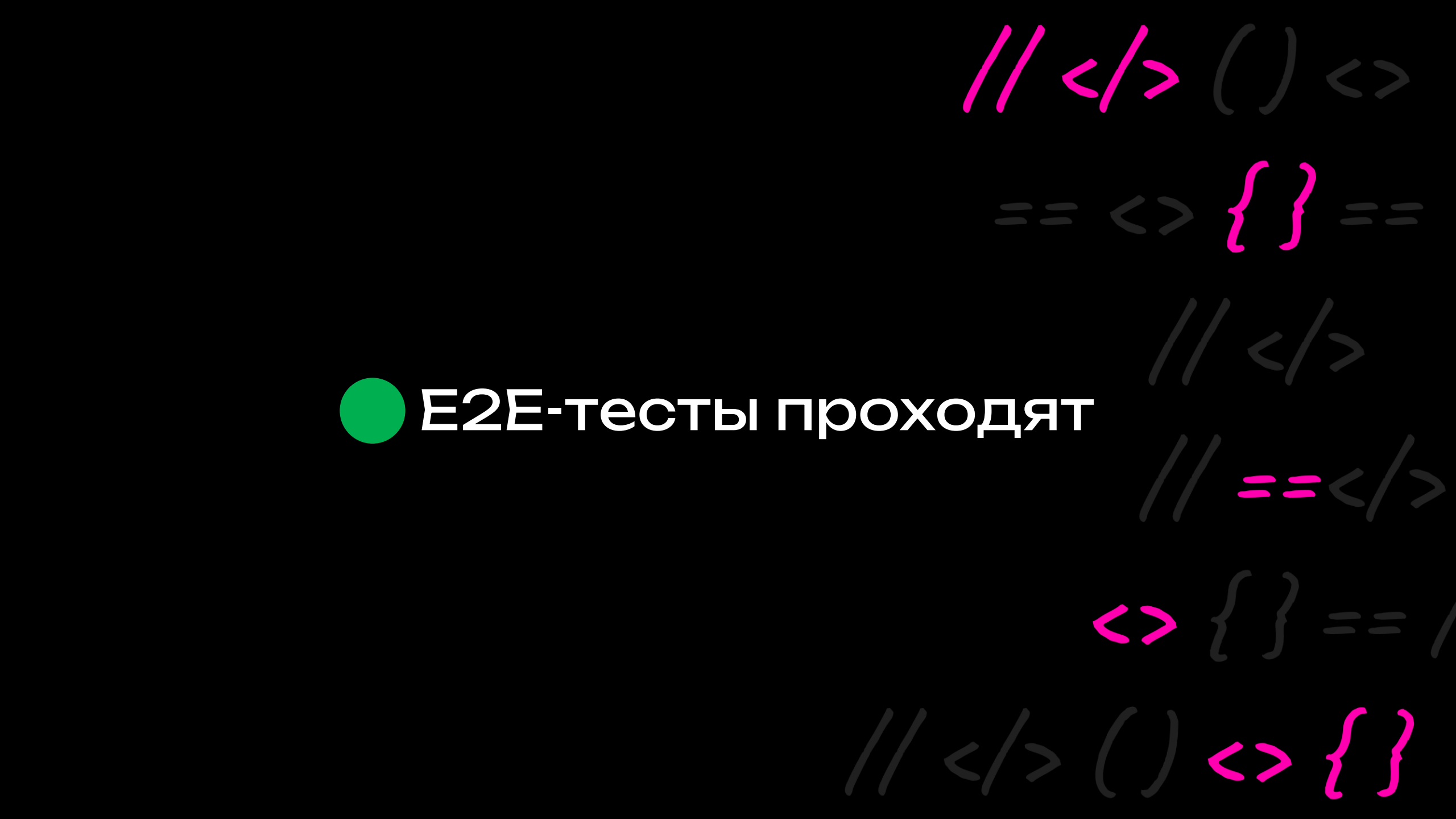




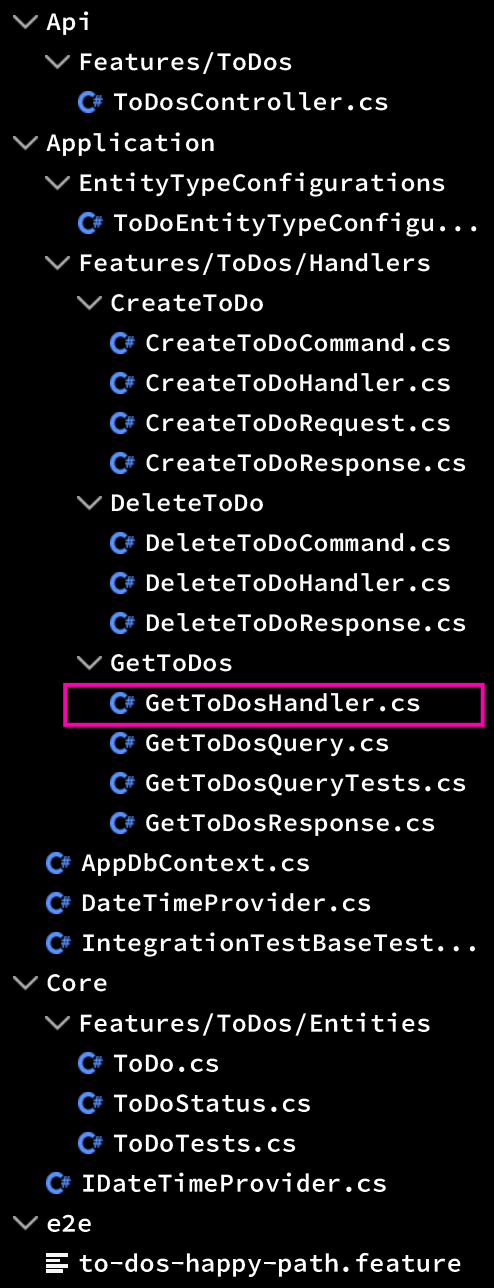
Минимально необходимое количество




```
public class DateTimeProvider : IDateTimeProvider
{
    public DateTime UtcNow => DateTime.UtcNow;
}
```

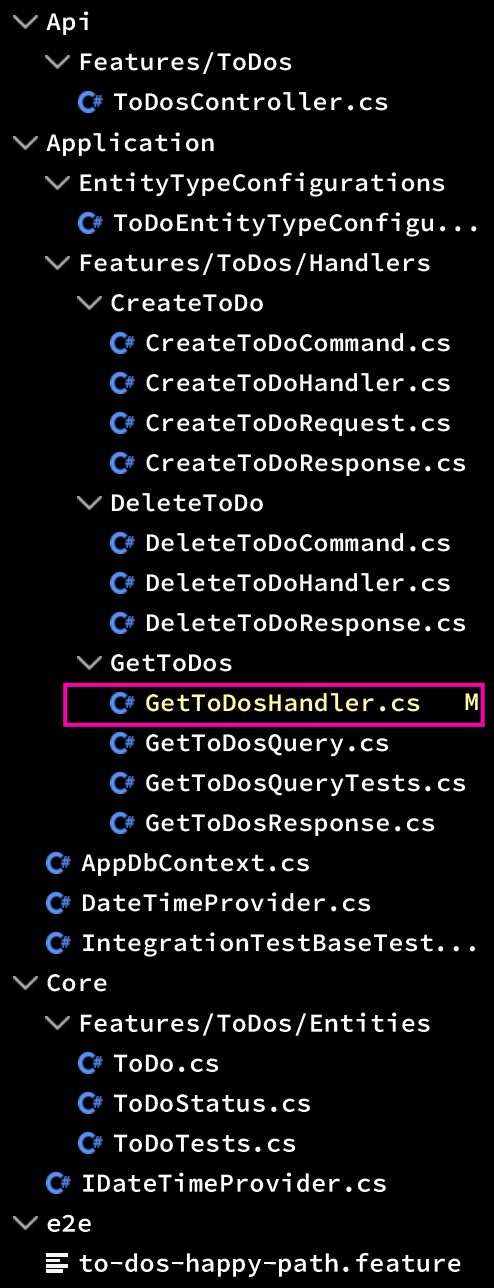

E2E-тесты проходят

И это плохо!

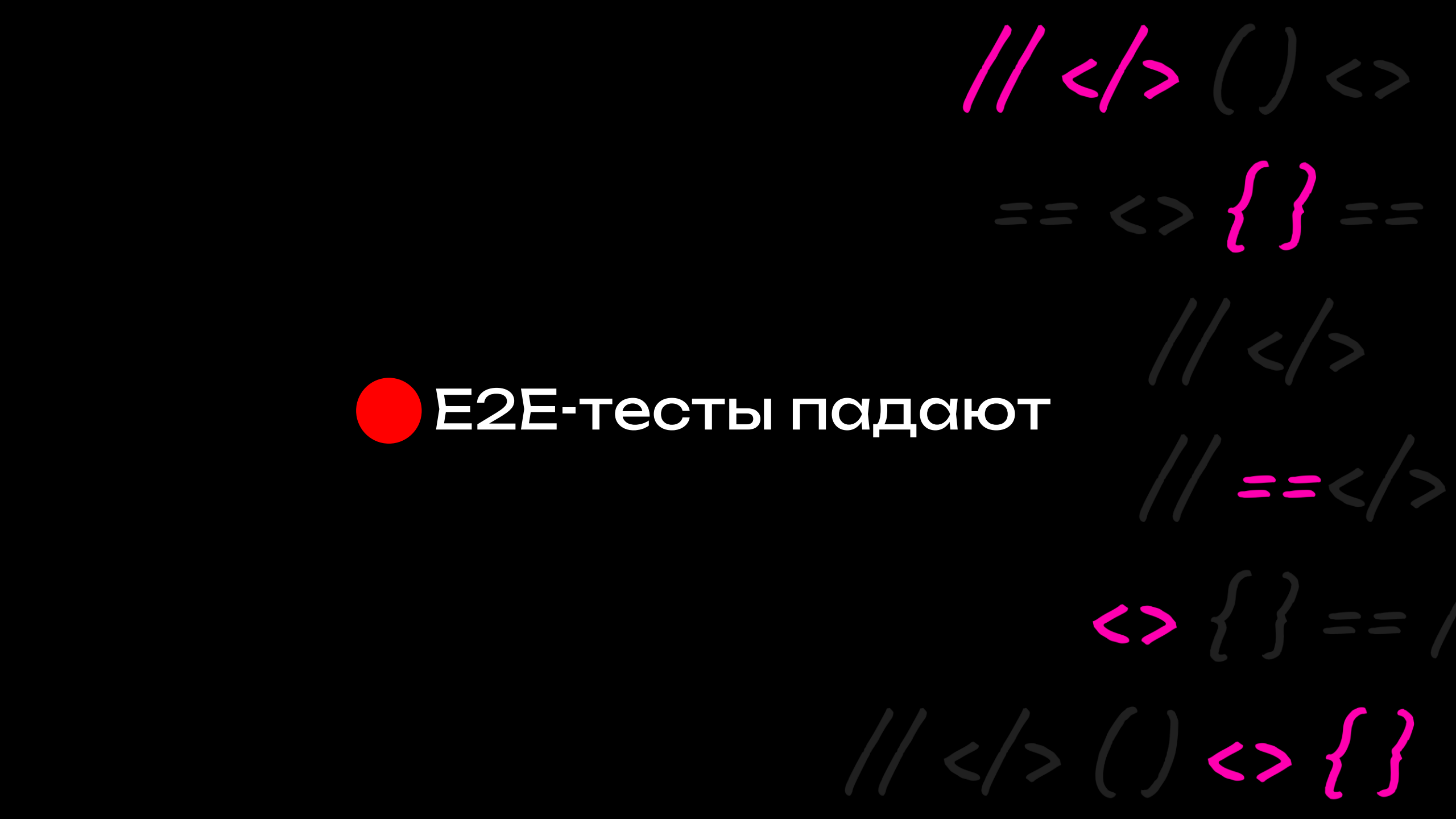



```
return new GetToDosResponse
{
  ToDos = toDos
    .Select(x => new ToDoDto
    {
      Id = x.Id,
      Name = x.Name,
      Status = ToDoStatus.New,
    })
    .ToList(),
};
```

Надо было всё ломать по TDD через None!




```
return new GetToDosResponse
{
  ToDos = toDos
    .Select(x => new ToDoDto
    {
      Id = x.Id,
      Name = x.Name,
      Status = x.GetStatus(dateTimeProvider),
    })
    .ToList(),
};
```


E2E-тесты падают

И это хорошо!

Karate вывод

{"id":80,"name":"[API-E2E]-to-do-0.00210826300845878 83","status":"Forgotten"}
{"id":80,"name":"[API-E2E]-to-do-0.00210826300845878 83","status":"New"}

```
$[0].status | not equal (STRING:STRING)
  'Forgotten'
  'New'
```

Не понятное


```
{
  "id": 80,
  "name": "[API-E2E]-to-do-0.0021082630084587883",
  "status": "Forgotten"
}
```

Что внутри?

CreatedAtUtc содержит DateTime {01/01/0001 00:00:00}

В базе created_at_utc timestamp with timezone

А там лежит -infinity




```
public class CreateToDoCommand(AppDbContext context)
{
  public async Task<long> ExecuteAsync(
    CreateToDoRequest createToDoRequest
  )
  {
    var newToDo = new ToDo {
      Name = createToDoRequest.Name,
    };

    await context.ToDos.AddAsync(newToDo);

    await context.SaveChangesAsync();

    return newToDo.Id;
  }
}
```




```
public class ToDo
{
    public ToDo() { }

    public long Id { get; set; }

    public required string Name { get; set; }

    public DateTime CreatedAtUtc { get; set; }
}
```




```
public class ToDo
{
    public ToDo() { }

    public long Id { get; set; }

    public required string Name { get; set; }

    public required DateTime CreatedAtUtc { get; set; }
}
```




```
public async Task<long> ExecuteAsync(
  CreateToDoRequest createToDoRequest
)
{
  var newToDo = new ToDo
  {
    Name = createToDoRequest.Name,
  };

  await context.ToDos.AddAsync(newToDo);

  await context.SaveChangesAsync();

  return newToDo.Id;
}
```

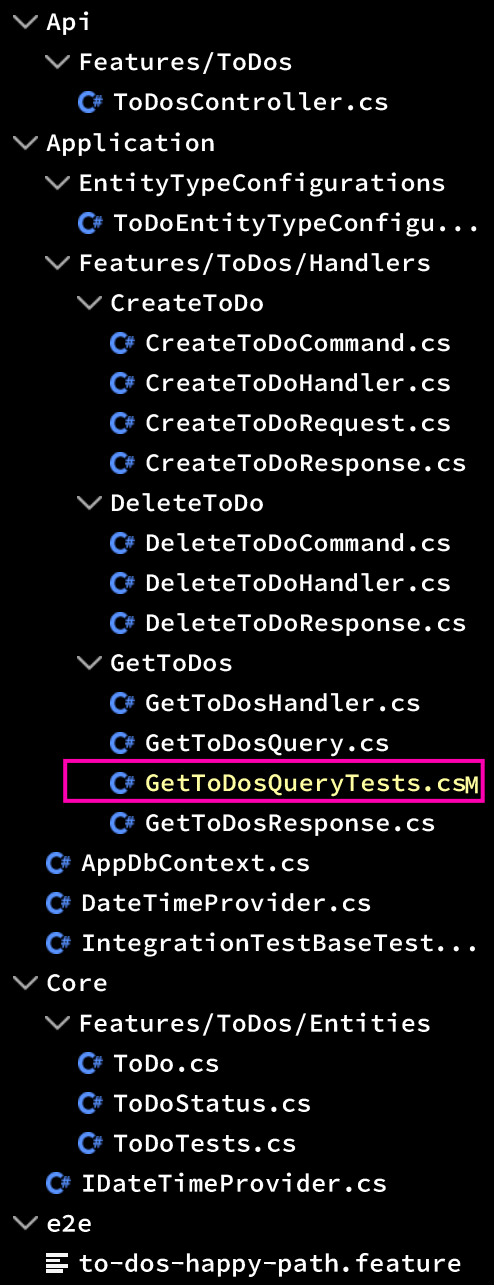



```
public async Task<long> ExecuteAsync(
  CreateToDoRequest createToDoRequest
)
{
  var newToDo = new ToDo
  {
    Name = createToDoRequest.Name,
    CreatedAtUtc = dateTimeProvider.UtcNow,
  };

  await context.ToDos.AddAsync(newToDo);

  await context.SaveChangesAsync();

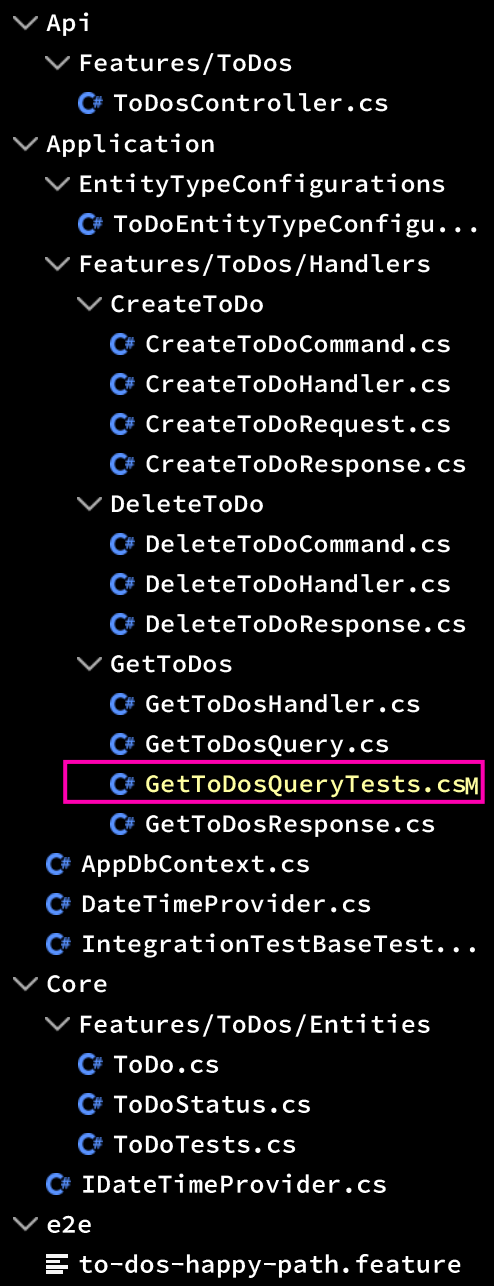
  return newToDo.Id;
}
```



GetAsync_ShouldOrderNewerToDosFirst

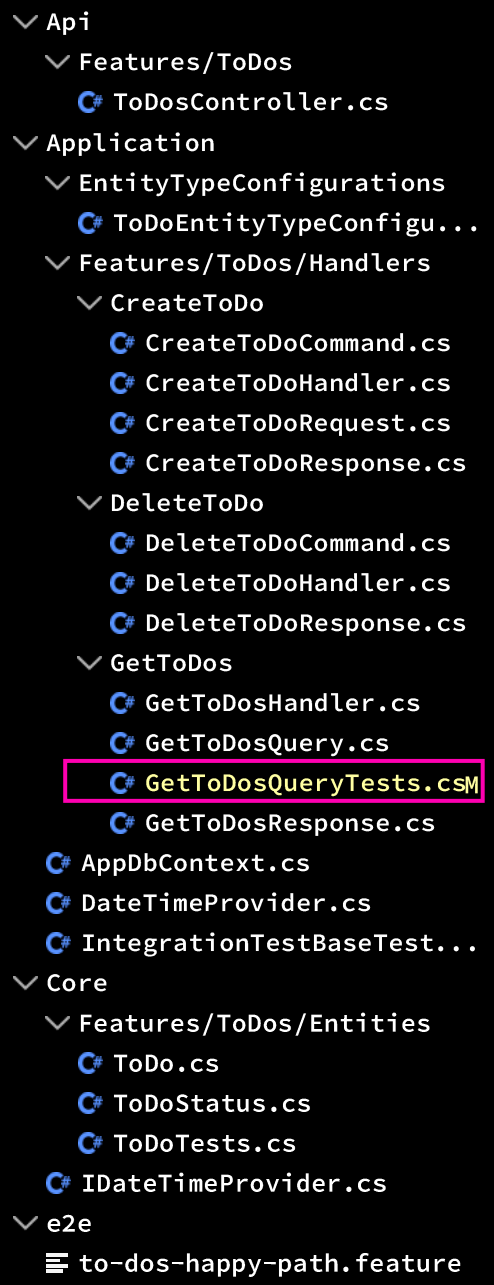
```
var dt = new DateTime(
    2026, 01, 02, 03, 04, 05, 678, DateTimeKind.Utc
);

var firstToDo = new ToDo
{
    Name = "First",
    CreatedAtUtc = dt.AddHours(1),
};
await AddEntityAndSaveAsync(context, firstToDo);
```



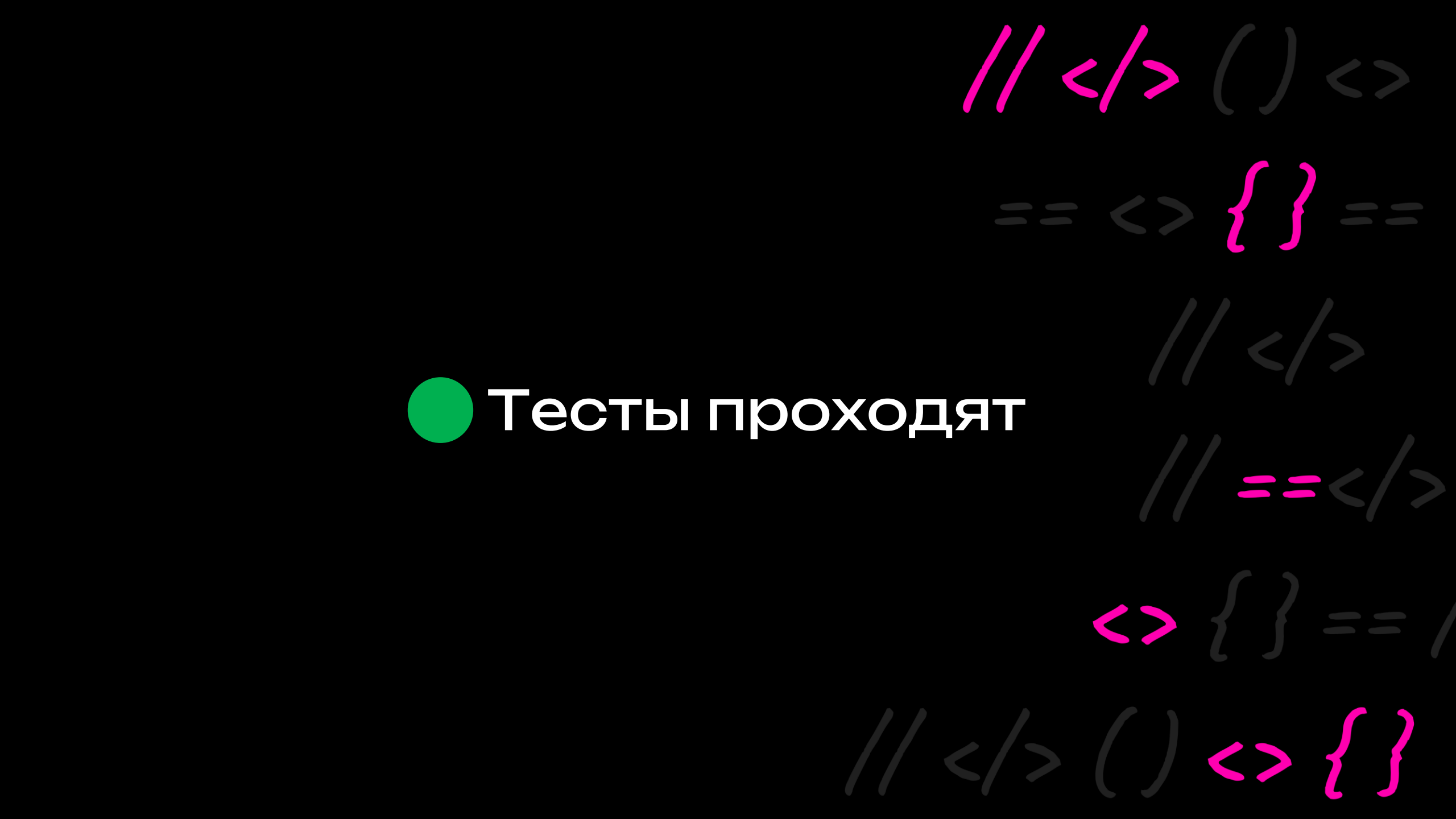
GetAsync_ShouldOrderNewerToDosFirst

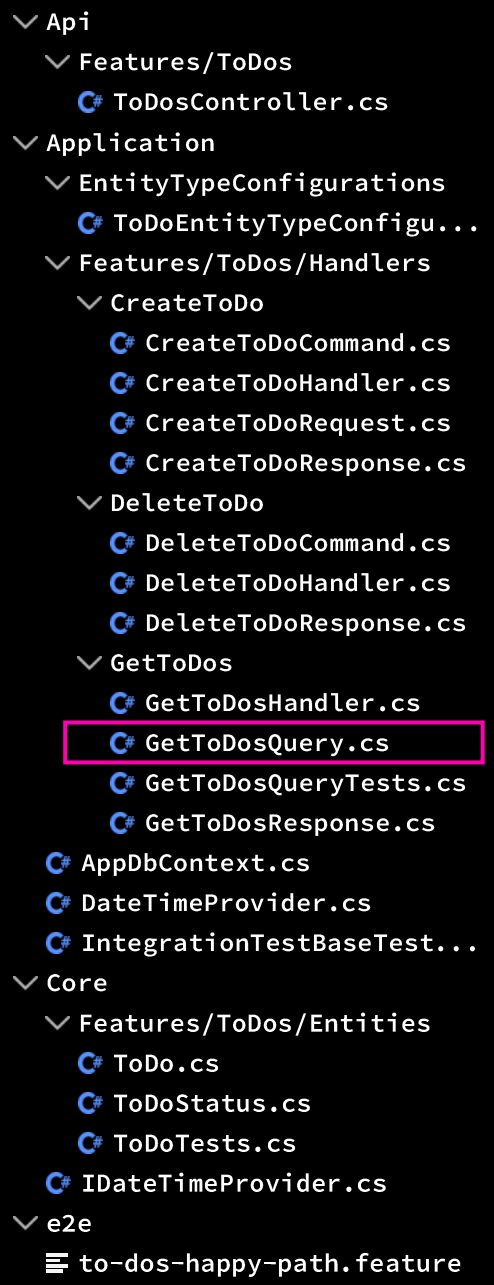
```
var secondToDo = new ToDo
{
    Name = "Second",
    CreatedAtUtc = dt.AddHours(2),
};
await AddEntityAndSaveAsync(context, secondToDo);
```



GetAsync_ShouldOrderNewerToDosFirst

```
var thirdToDo = new ToDo
{
    Name = "Third",
    CreatedAtUtc = dt.AddHours(3),
};
await AddEntityAndSaveAsync(context, thirdToDo);
```


Тесты проходят




```
public class GetToDosQuery(AppDbContext context)
{
  public Task<List<ToDo>> GetAsync()
  {
    return context
      .QueryableAsNoTracking<ToDo>()
      .OrderByDescending(x => x.Id)
      .ToListAsync();
  }
}
```




```
public class GetToDosQuery(AppDbContext context)
{
  public Task<List<ToDo>> GetAsync()
  {
    return context
      .QueryableAsNoTracking<ToDo>()
      .OrderByDescending(x => x.CreatedAtUtc)
      .ToListAsync();
  }
}
```

- Тесты всё ещё проходят

Тесты на контроллер!

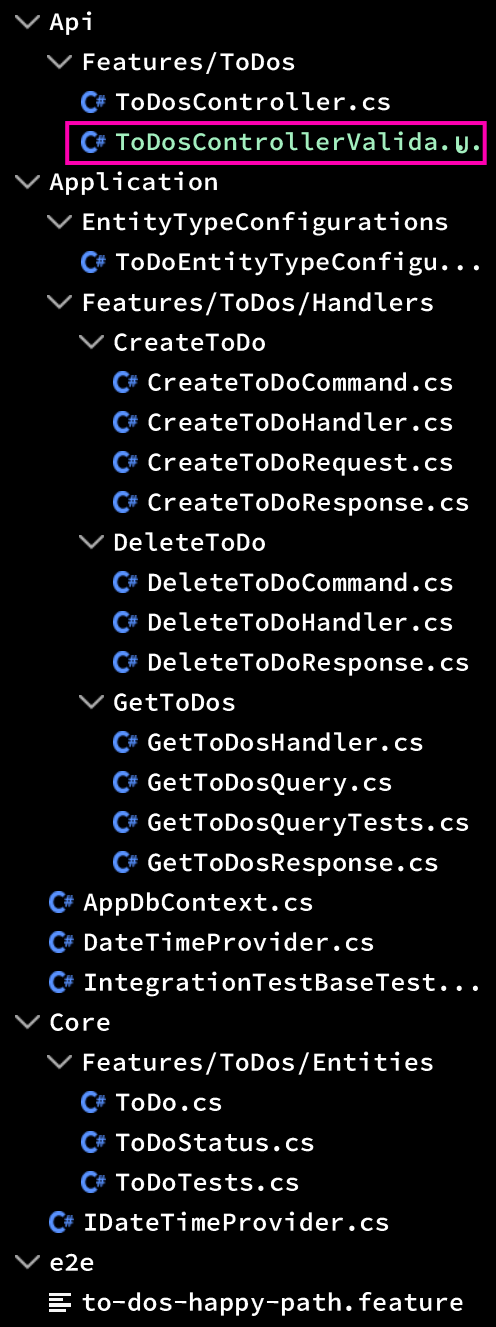



```
public class ToDoEntityTypeConfiguration
  : IEntityTypeConfiguration<ToDo>
{
  public void Configure(EntityTypeBuilder<ToDo> builder)
  {
    builder.HasIndex(x => x.Id).IsUnique();

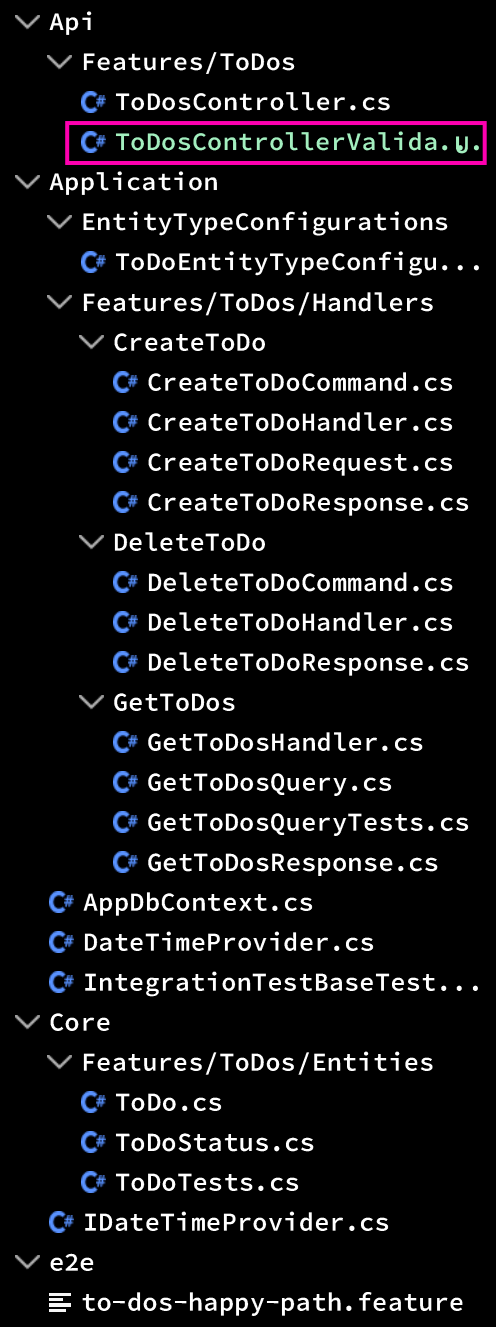
    builder.Property(x => x.Name).IsRequired()
      .HasMaxLength(256);
  }
}
```




```
public static IEnumerable<object[]>
  CreateToDoRequestInvalidNameTestCases =>
  [
    [null!],
    [""],
    [new string('x', 257)],
  ];
```



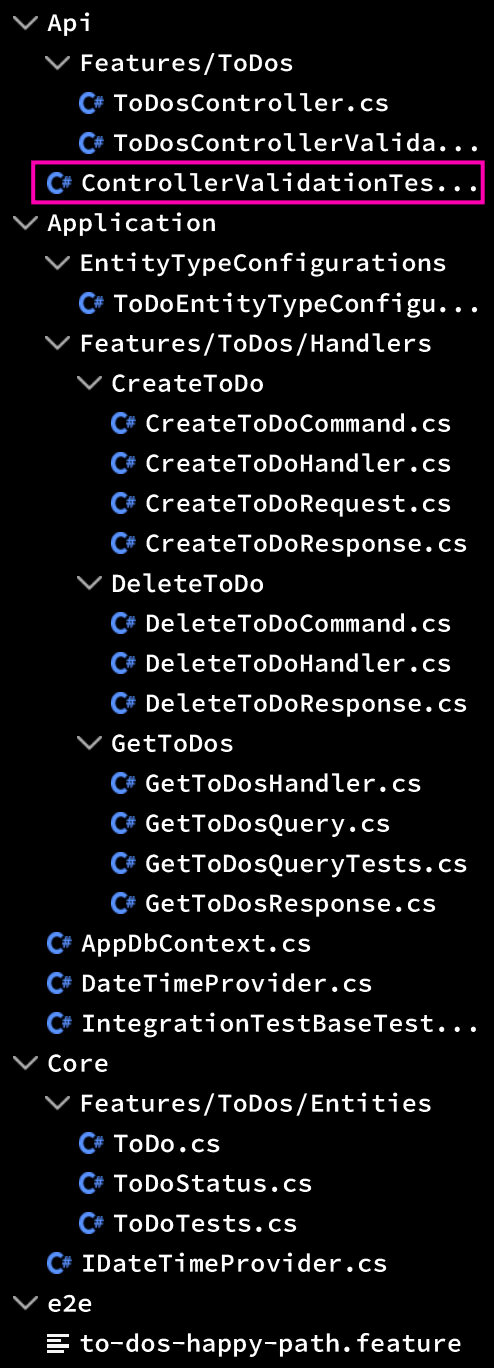

```
public class ToDosControllerValidationTests(
  WebApplicationFactory<Program> factory
) : ControllerValidationTestsBase(factory)
{
  [Theory]
  [MemberData(nameof(
    CreateToDoRequestInvalidNameTestCases
  ))]
  public async Task
    CreateToDoAsync_ShouldThrowBadRequestIfNameIsInvalid(
      string name
    )
  {
```




```
    var createToDoRequest = new CreateToDoRequest {
      Name = name,
    };

    var response = await _httpClient.PostAsJsonAsync(
      "/to-dos",
      createToDoRequest
    );

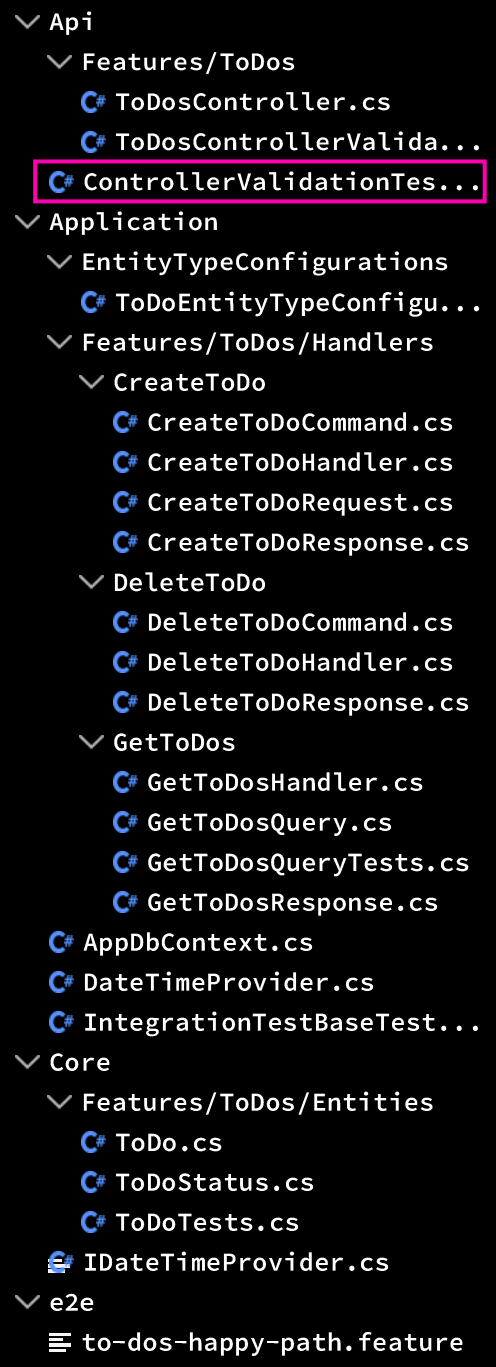
    Assert.NotNull(response);
    Assert.Equal(
      StatusCodes.Status400BadRequest,
      (int)response.StatusCode
    );
  }
```




```
public class ControllerValidationTestsBase(
  WebApplicationFactory<Program> factory
) : IClassFixture<WebApplicationFactory<Program>>,
    IAsyncLifetime
{
  protected HttpClient _httpClient = null!;

  public async Task InitializeAsync()
  {
    _httpClient = factory.CreateClient();

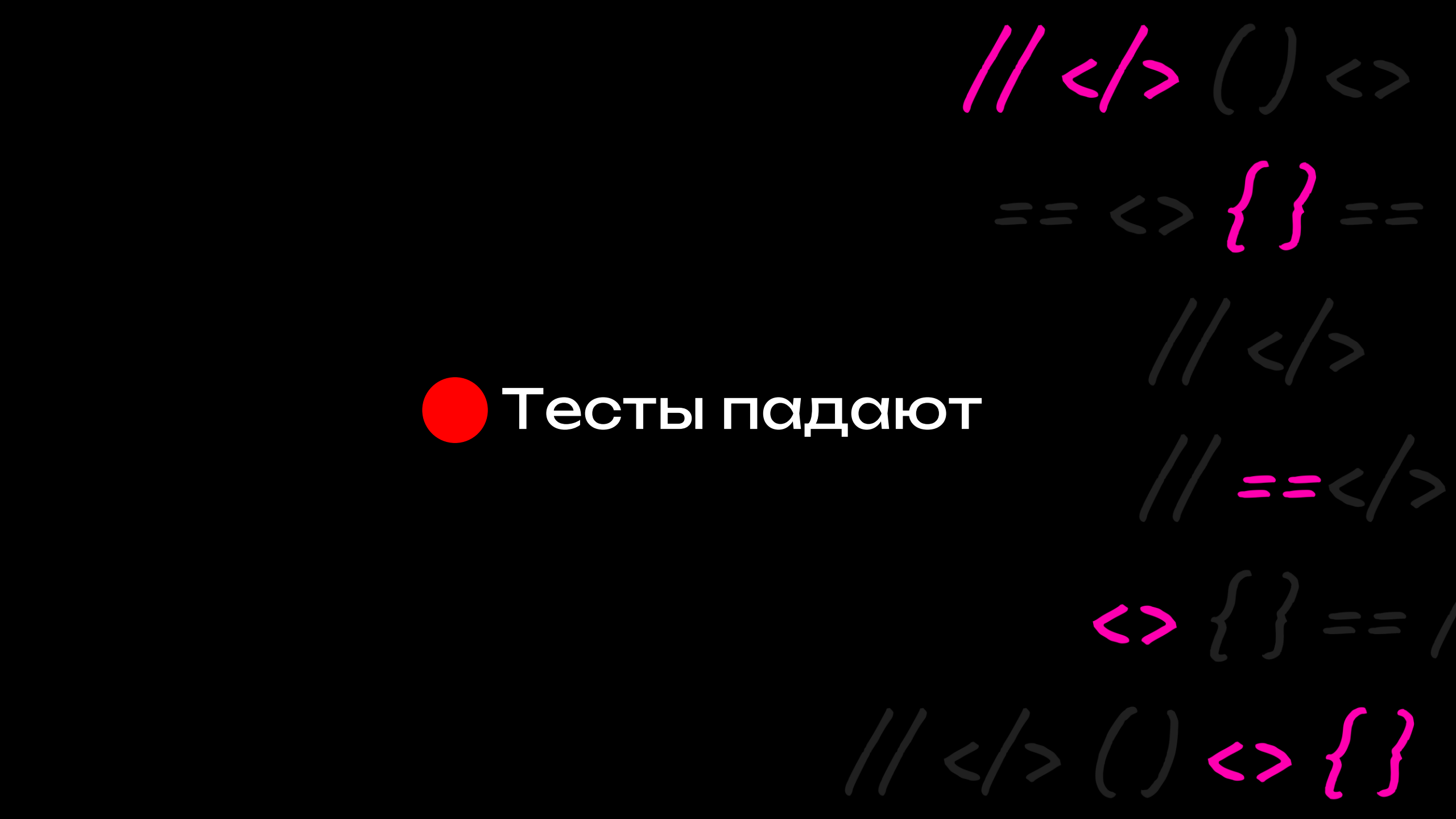
    factory = factory.WithWebHostBuilder(builder =>
    {
```

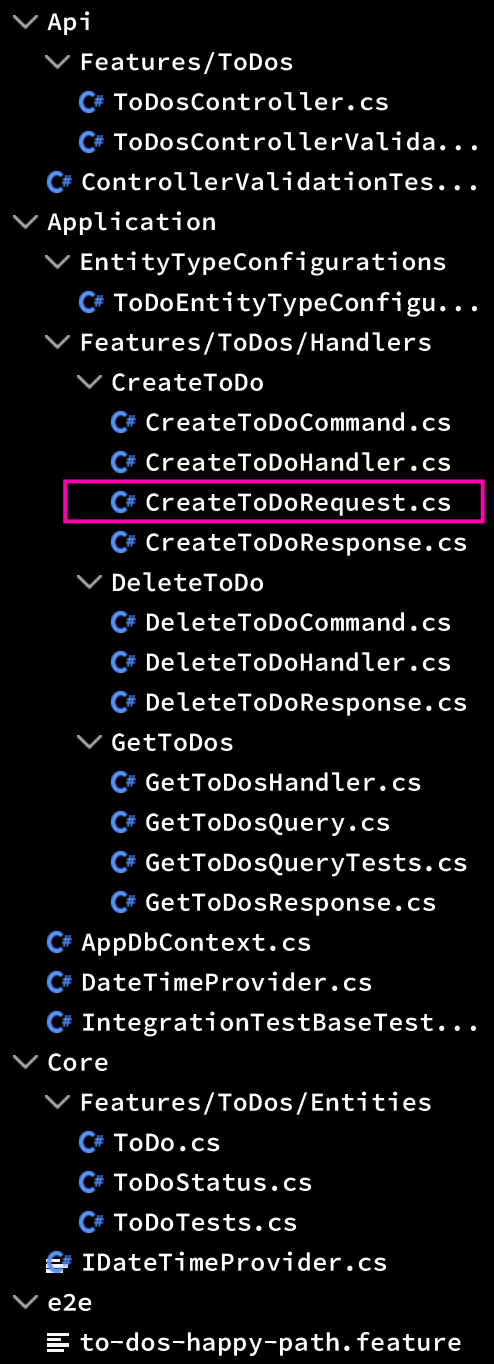



```
services.Remove(
  services.Single(x =>
    x.ServiceType
    == typeof(
      IDbContextOptionsConfiguration<AppDbContext>
    )
  )
);

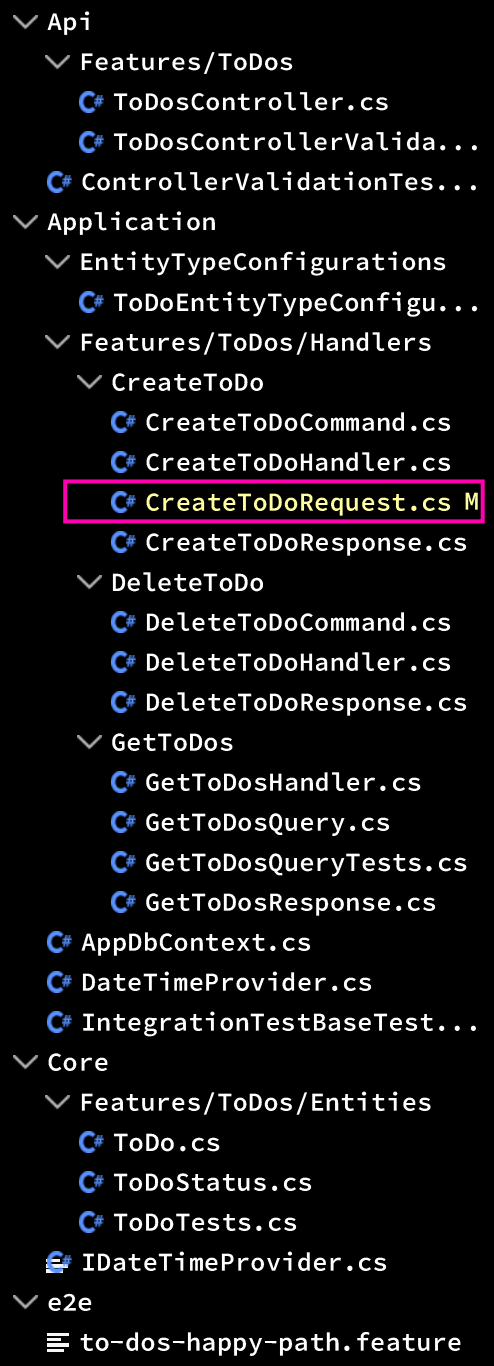
services.AddDbContext<AppDbContext>(options =>
  options.UseInMemoryDatabase(
    databaseName: new Random().Next().ToString()
  )
);
```

.NET красавка!

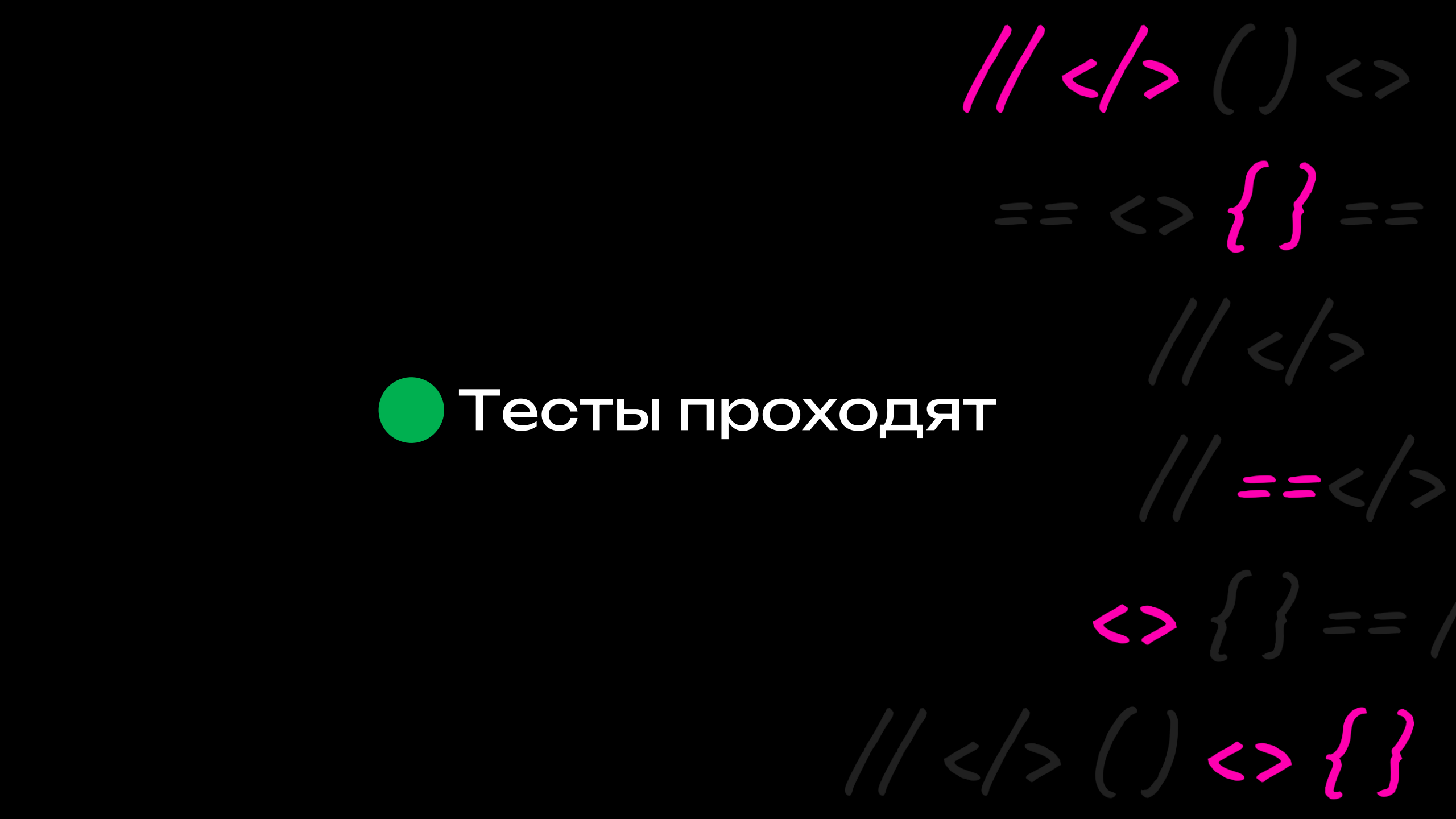

Тесты падают




```
public class CreateToDoRequest
{
    public required string Name { get; set; }
}
```




```
public class CreateToDoRequest
{
    [Required]
    [MaxLength(256)]
    public required string Name { get; set; }
}
```


Тесты проходят

Саммари и расход!

Тесты кто, где, с кем, зачем?

E2E

Happy Path тесты.

Минимум краевых случаев.

Реальная база.

Integrational

Сортировка, фильтрация, краевые случаи, валидации на уровне базы.

Реальная база.

Modular

Бизнес-правила, расчёты, не зависящие от ORM.

Controller

Валидация входа.

In-memory база.

Handler

Тестирование флоу вызовов команд, квери, мапперов, калькуляторов и т.д.

Новая папка

Здесь будет ваша придумка.

Выводы / Тезисы / Мысли

- ✓ Можно придумать как протестировать
- ✓ Код тестов как код сервиса
- ✓ TDD не сложно, если архитектура и инфраструктура его поддерживают
- ✓ TDD влияет на тактический дизайн кода
- ✓ TDD не влияет на дизайн кода, если есть архитектура и опыт в ней для типовых задач

tourmaline
core

.NET бэкенд по TDD

С НУЛЯ ДО PRODUCTION-READY

Анонс серии видео



Здесь будут видео,
как сделать такое
с нуля

Ссылка на плейлист

tourmaline
core

Основной репозиторий
на .NET 10



to-dos-api-dotnet

Production-ready пример
на .NET 9



inner-circle-time-api

Нюансы DexExp





Сетап проекта, пайплайнов и всего DevExp

Ссылка на видео